

RSG Policy Reconciliation — Complete Requirements Package

Version 1.0 · 2026-08-18 · Risk Solutions Group · Zoho Creator · Zia AI

This is the only document Zia needs. It contains the PRD, BRD, RFP, process diagrams, field dictionaries, picklists, verdicts, scoring table, Deluge, and seed acceptance cases.

1. Create app `rsg_policy_reconciliation`.
2. Build **Phase 1 only**: five forms, all fields, picklists, lookups, status-history hook, Gretchen vs Lamar permissions.
3. **Stop**. Do not start Phase 2 until the issuer says so.
4. Do not invent fields, verdicts, policy numbers, premiums, or GUIDs.
5. Never write NowCerts. Never create Zoho CRM Accounts.

Five forms in this order: `Policy_Master` · `Policy_Status_History` · `Renewal_Queue` · `Policy_Audit` · `Audit_Exceptions`.

0. Build prompt (Zia)

Zia prompt — paste this first

You are building a Zoho Creator application for Risk Solutions Group (RSG) called **RSG Policy Reconciliation**.

You are attaching **one document**: ZIA_UPLOAD.pdf. It is the complete package (PRD + BRD + RFP + process diagrams + field dictionaries + Deluge).

Build Phase 1 only, then stop.

Read those uploads as the only source of truth (zia_prompt , specification_markdown , forms , picklists , deluge , sample_records). Build the application **exactly** as specified.

Hard rules

1. Create **exactly five forms**, with these API names, in this order:
2. Policy_Master
3. Policy_Status_History
4. Renewal_Queue
5. Policy_Audit
6. Audit_Exceptions
7. Create **every field** listed in the form sheets/CSVs. Do not invent extra fields. Do not drop required fields. Use Deluge_Name as the Creator field link name.
8. Use picklist values **exactly** as listed. Do not paraphrase, merge, or retitle options.
9. Implement the **12 verdict types** and the **0-100 confidence score** using the Deluge in the pack. Do not substitute a different scoring model.
10. **NowCerts is the system of record for policy facts**. Zoho CRM is the system of record for Accounts/Contacts/Deals. This Creator app never writes to NowCerts. It never writes to Zoho CRM unless a human set Approved_To_Push = true **and** Approved_By + Approved_At are filled.
11. Cancellations must be classified as one of: Non Pay , Rewrite , Insured Request , Underwriter , Other . Rewrites must link Rewrite_Of / Successor_Policy .
12. Build **Phase 1 only** first (forms, fields, relationships, picklists, status-history hook). Stop and wait for confirmation before Phase 2.
13. Users: **Lamar** = Admin / Producer. **Gretchen** = CSR. Do not give Gretchen permission to approve AMS/CRM pushes or to close High/Critical exceptions without Lamar.

Success looks like

After Phase 1, a user can:

- Add a Policy_Master row with Policy_Number (NowCerts_Policy_GUID when known).
- Change Policy_Status and automatically get a Policy_Status_History row.
- Open the five forms, their lookups, and the Admin / CSR views listed in the Views sheet.

Do not generate sample insurance data with invented policy numbers, premiums, or carrier quotes. Use only Sample_Policy_Master / sample_records if seed data is needed.

PRD — Product Requirements Document

Product Requirements Document (PRD)

Product: RSG Policy Reconciliation

Platform: Zoho Creator

Customer: Risk Solutions Group (RSG)

Version: 1.0

Date: 2026-08-18

Owner: Lamar (Operations / Producer)

Builder: Zia AI (Zoho Creator)

Status: Approved for Phase 1 build

1. Problem

RSG's policy truth lives in **NowCerts (AMS)**. Accounts and deals live in **Zoho CRM**. Renewals and ops state live in **Hermes / Supabase**. Those copies drift.

Today there is no single desk that:

- compares AMS vs CRM vs the renewal worklist on every policy
- explains *why* they disagree (verdict)
- scores how trustworthy the comparison is (0–100)
- opens an exception with an SLA instead of a Slack guess
- classifies cancellations (Non Pay vs Rewrite vs Insured Request vs Underwriter)
- links rewrites instead of treating them as lost business

The agency cannot scale to the \$1M premium north star if Gretchen and Lamar reconcile the book by hand.

2. Product statement

Build a Zoho Creator application named **RSG Policy Reconciliation** (`rsg_policy_reconciliation`) that is the **reconciliation workspace** for the in-force and recently cancelled book.

It is **not** a CRM. It is **not** an AMS. It does **not** book commissions.

For every policy it must emit:

1. Exactly one of **12 verdicts**

2. An integer **confidence score 0–100**
3. A written **recommendation**
4. An **Audit_Exception** when the verdict is not `clean_match` (or confidence < 80, or premium delta exceeds the hybrid threshold)

3. Goals

ID	Goal	Measure
G1	One working row per AMS policy GUID	Unique <code>NowCerts_Policy_GUID</code> on <code>Policy_Master</code>
G2	Deterministic recon, not an LLM at runtime	<code>Deluge thisapp.recon.verdict / score</code> only
G3	Cancellation class always present on cancel	Save blocked if class empty (except import seeds)
G4	Rewrites linked, not double-counted	<code>Rewrite_Of / Successor_Policy_Number</code>
G5	Stale renewals visible same day in 30-day bucket	<code>stale_renewal_queue</code> + CSR view
G6	No silent writes	CRM update only with approval gate; zero NowCerts writes

4. Non-goals

- Creating Zoho CRM Accounts
- Writing NowCerts / Momentum (binds, endorsements, insured inserts)
- Commission ledger / money booking (Hermes `commission_ledger` stays SoR)
- Client-facing portal
- Inventing premiums, policy numbers, GUIDs, or carrier quotes
- A second 0–100 *renewal risk* score (Risk_Status stays SAFE / AT_RISK / CRITICAL / RENEWED / LAPSED)

5. Users

Persona	Creator role	Jobs to be done
Lamar	Admin / recon-approver	Run recon, approve CRM drafts, close High/Critical exceptions, see Tier A money drift
Gretchen	CSR	Classify cancels, work stale renewals, resolve Low/Medium exceptions, add notes
Hermes	Integration user	Scheduled CRM pull; cannot set <code>Approved_To_Push</code>

6. User stories

1. **As Lamar**, I run daily recon at 6:45am ET and see counts by verdict, not a dump of every policy.
2. **As Gretchen**, I open CSR Queue and only see Low/Medium exceptions assigned to me.
3. **As Gretchen**, when a policy is Cancelled I must pick Non Pay, Rewrite, Insured Request, Underwriter, or Other before save.
4. **As Lamar**, I never want Creator to push CRM until I set Approved_To_Push, Approved_By, and Approved_At (within 24 hours).
5. **As Gretchen**, opening a renewal record does **not** count as a touch; only real field edits stamp Last_Touched_At.
6. **As Lamar**, if AMS has a policy and CRM does not, the app drafts a Policy create and **does not** create an Account.
7. **As either**, if two Policy_Master rows share a Policy_Number, verdict is duplicate_policy — uniqueness is **not** enforced on Policy_Number so the duplicate can be stored and flagged.

7. In-scope product surface

7.1 Five forms (exact API names, this order)

1. Policy_Master — working copy (67 fields; see field CSV)
2. Policy_Status_History — immutable status audit
3. Renewal_Queue — Project 85 style worklist
4. Policy_Audit — one row per run per policy
5. Policy_Audit lookup child: Audit_Exceptions — SLA work items

Do not add a sixth form in Phase 1.

7.2 Twelve verdicts (do not rename)

duplicate_policy → pending_sync → rewrite_detected → status_mismatch → financial_discrepancy → missing_in_crm → missing_in_ams → stale_renewal_queue → stale_crm → cancel_reason_gap → lineage_orphan → clean_match

First match wins.

7.3 Confidence

Start at 100. Subtract per scoring table S1–S25. Integer. Floor 0, ceiling 100. Write Score_Breakdown.

7.4 Financial rule

Flag `financial_discrepancy` when `abs(Premium - CRM_Premium) ≥ $25` **OR** `abs(percent) ≥ 1%`. `Agency_Fee` is not premium.

7.5 Renewal risk (separate from confidence)

Premium increase first, then timing: >15% CRITICAL, ≥5% AT_RISK; else ≤30 days CRITICAL, 31–90 AT_RISK, else SAFE.

8. Phased delivery

Phase	Product increment	Zia stop rule
1	Schema, picklists, lookups, permissions, status-history hook, Policy_Tier	STOP. Wait.
2	Verdict + score + rewrite + Run Recon button + sample seeds	Stop for acceptance
3	Daily/weekly schedules, SLA sweep, views, Recon Desk, email	
4	Zoho CRM read; gated write stub	
5	Optional Hermes health HTTP	
6	Zoho Analytics + harden	

9. Success metrics (product)

- Phase 2: all 14 seed policies in `sample_records.json` match expected verdict **and** confidence.
- Phase 3: no second Open exception for the same policy + verdict.
- Phase 4: Deluge contains no `zoho.crm.createRecord("Accounts")`.
- Production: Gretchen cannot resolve High/Critical; cannot set `Approved_To_Push`.

10. Constraints Zia must obey

1. NowCerts wins policy facts (premium, dates, carrier, status).
2. Unknown AMS status → `Sync_Status=Error`, do not invent a picklist value.
3. Mid-term cancel: keep `Expiration_Date`; store cutoff in `Cancellation_Date`.
4. Pending Cancel is **not** auto-excluded from the renewal queue (`needs_verification`).
5. Non Pay is not a rewrite unless a replacement policy is identified.
6. Time zone America/New_York. Currency USD.

11. Dependencies

Field-level create lists: `forms_*.csv`, `picklists.csv`, `views.csv`, `workflows.csv`, `deluge/*.dg`. Those files are part of this PRD's data appendix (also inside `ZIA_UPLOAD.xlsx`).

12. Open questions (do not invent answers)

- Live Zoho CRM Policies module API name after org create (may be CustomModuleX). Confirm before Phase 4.
- Gretchen and Lamar Zoho user emails for Owner assignment (placeholders `gretchen` / `lamar` until replaced).

BRD — Business Requirements Document

Business Requirements Document (BRD)

Initiative: Insurance Policy Reconciliation Agent

Organization: Risk Solutions Group (RSG)

Application: Zoho Creator — RSG Policy Reconciliation

Version: 1.0

Date: 2026-08-18

Sponsor: Lamar, Operations

Prepared for: Zia AI (Zoho Creator) build

1. Business purpose

Protect retention (Project 85, target 75%+) and book integrity by making AMS / CRM / renewal-queue disagreement **visible, classified, and owned** every business day.

Drift today causes:

- false tombstones (CRM or mirror says gone; AMS still has the policy)
- unclassified cancellations (Non Pay treated like Rewrite, or the reverse)
- stale 30-day renewals with no recorded touch
- premium in CRM that does not match NowCerts
- duplicate policy rows that inflate or hide the book

2. Stakeholders

Stakeholder	Interest	Authority
Lamar	Book, money, High/Critical exceptions, CRM push approval	Product owner / Admin
Gretchen	Daily CSR work: cancels, renewals, Low/Medium exceptions	CSR user
Hermes (system)	Scheduled CRM pull, optional health read	Integration; no approvals
NowCerts	Policy system of record	Not written by this app
Zoho CRM	Account / Deal / Policy mirror	Read always; write only when approved

3. Current state vs future state

Current	Future
Operators compare AMS, CRM, and renewal lists by memory and exports	Creator <code>Policy_Master</code> holds the working copy; agent compares snapshots
Status changes are not a durable trail	Every status/Active change inserts <code>Policy_Status_History</code>
Cancel reason is free text or missing	Required <code>Cancellation_Class</code> enumeration
Rewrites look like lost policies	Heuristic + human link <code>Rewrite_Of</code> / successor
Stale renewals discovered late	Queue stale rules (7/14 day by cadence) + CSR view
CRM updates happen ad hoc	Draft payload on <code>Policy_Audit</code> ; execute only through approval gate

4. Systems of record (business rule)

Domain	Winner	This app
Policy facts (status, premium, dates, carrier, GUID)	NowCerts	Copy + flag CRM lag
Accounts, Contacts, Deals	Zoho CRM	Stamp IDs only; never create Accounts
Renewal eligibility / Project 85 ledger	Hermes / Supabase	Creator queue is a worklist , not a second eligibility engine
Commissions	Hermes money tables	Flag premium drift only; do not book
Notes, assignment, last contact	Human in Creator/ CRM	AMS pull must not overwrite Notes

Conflict table (must implement):

- AMS vs CRM on premium/dates/carrier/status → NowCerts wins; verdict `stale_crm` or `status_mismatch`; draft CRM update
- Duplicate number or GUID → `duplicate_policy`; human merge
- Outbound sync still pending → `pending_sync`; do not re-push
- Cancelled + overlapping new policy same insured+LOB → `rewrite_detected`
- CRM has policy, AMS GUID empty → `missing_in_ams`; **do not tombstone**
- AMS has policy, CRM ID empty >24h → `missing_in_crm`; no Account create

5. Business requirements

ID	Requirement	Priority
BR-01	Maintain one Policy_Master row per NowCerts policy GUID	Must
BR-02	Policy_Number is not database-unique so duplicates can be recorded and flagged	Must
BR-03	Normalize AMS/CRM status and billing to the RSG picklists; unknown status is Error	Must
BR-04	Classify every cancel: Non Pay, Rewrite, Insured Request, Underwriter, Other	Must
BR-05	Detect rewrites: same insured GUID + same LOB + successor effective within 60 days of cancel/expiration	Must
BR-06	If multiple successor candidates, cap confidence at 60 and open High exception — human picks the link	Must
BR-07	Hybrid premium discrepancy: $\geq \$25$ OR $\geq 1\%$	Must
BR-08	Agency_Fee is shop fee, not commission; missing Agency Bill fee is info, not financial_discrepancy alone	Must
BR-09	Renewal queue stale: 7 days in 30/Past Due Open or In Progress; 14 days in 60 Open; or policy left Open after terminal status; or expiration dates disagree	Must
BR-10	Last_Touched_At updates only on Queue_Status, Last_Contact_Date, Strategy_Notes, Risk_Status — not form open	Must
BR-11	Never hard-delete Policy_Master or Renewal_Queue; dismiss = checkbox + status	Must
BR-12	Exception SLA (calendar hours v1): Critical 4, High 24, Medium 72, Low 120	Must
BR-13	Auto-close Open exceptions on later clean_match only if prior verdict was pending_sync or stale_crm	Must
BR-14	Gretchen cannot resolve High/Critical; cannot approve CRM push	Must
BR-15	Policy_Tier: A if annualized $\geq \$5000$ or Commercial premium $\geq \$2500$; B if Active or expiring ≤ 90 days; else C	Must
BR-16	Daily recon 6:45am ET for A/B/Active/120-day window/dirty verdicts; weekly Sunday 7:00am for Tier C	Must
BR-17	No NowCerts write APIs from Creator	Must
BR-18	No Zoho CRM Account create; Policy create is a human-executed draft	Must

6. Process requirements

See attached process diagrams (same package):

1. Systems of record
2. Daily reconciliation

3. Verdict decision tree
4. Cancellation and rewrite
5. Exception and SLA
6. Approval-gated CRM write

7. Data requirements (forms)

Five forms only. Field dictionaries are the `forms_*.csv` files (appendix). Summary:

- **Policy_Master:** identity, insured snapshot, coverage, status, dates, financials, lineage, recon stamps, approval gate
- **Policy_Status_History:** source system AMS/CRM/Creator/Hermes; old/new status and Active
- **Renewal_Queue:** premiums current/renewal, Increase_Percent formula, Risk_Status, Cadence_Bucket, stale flags
- **Policy_Audit:** Run_ID, verdict, confidence, snapshots, findings vs recommendation, Recommended_Payload JSON
- **Audit_Exceptions:** severity, SLA clocks, resolution class

8. Assumptions

- Zoho CRM Policies custom module exists or will exist per `docs/zoho/fields_policies.csv`
- Gretchen and Lamar have Creator users
- Phase 1 has no live CRM connection
- Seed data is only `tests/sample_records.json` (prefixed RSG-TEST-)

9. Constraints

- Time zone America/New_York
- USD, 2 decimals
- Picklist strings are exact (including `Won't Fix`, `Non Pay`, `AT_RISK`)
- Deluge dialect may need minor syntax adjust; **logic and names must not change**

10. Acceptance (business)

Business accepts Phase 2 when each seed verdict/confidence in `acceptance_cases` is reproduced with no invented policies. Business accepts Phase 4 when a sandbox CRM push is blocked without the approval trio and Accounts are never created.

RFP — Statement of Work

Request for Proposal (RFP) — Build Response for Zia AI

RFP number: RSG-CREATOR-RECON-2026-01

Title: Zoho Creator application — RSG Policy Reconciliation

Issuer: Risk Solutions Group

Respondent: Zia AI (Zoho Creator)

Issue date: 2026-08-18

Response type: Build the application in Zoho Creator. Do not write a sales proposal. Do not invent scope.

This RFP is the statement of work. Attach it to Zia together with the PRD, BRD, and process-diagram PDF. Field CSVs and Deluge in `ZIA_UPLOAD.xlsx` are the data appendix and win over prose if they conflict.

1. Invitation

RSG invites Zia to **create** a Zoho Creator application that reconciles insurance policies across NowCerts (AMS), Zoho CRM, and the renewal worklist.

Success is a working Creator app that matches this RFP. Failure is extra forms, renamed verdicts, invented fields, LLM-based scoring, or any NowCerts write.

2. Scope of work

In scope

- Application display name: **RSG Policy Reconciliation**
- Application link name: `rsg_policy_reconciliation`
- Exactly **five** forms, API names in this order:
 - `Policy_Master`
 - `Policy_Status_History`
 - `Renewal_Queue`
 - `Policy_Audit`
 - `Audit_Exceptions`
- Every field in the form dictionary (appendix / Excel sheets)
- Every picklist value exactly as listed
- Lookups and related lists as specified

- Role permissions: Lamar Admin, Gretchen CSR
- Status-history hook on Policy_Master add/edit
- **Phase 1 only** until the issuer confirms Phase 2

Out of scope (reject if proposed)

- Additional forms in Phase 1
- NowCerts / Momentum POST, Insert, PartialUpdate
- `zoho.crm.createRecord` for Accounts
- Commission posting
- Client portal
- Renaming the 12 verdicts
- A generative model inside `thisapp.recon.verdict` or `score`

3. Mandatory technical requirements

ID	Requirement	Proof
M1	Deluge_Name in CSVs = Creator field link names	Export of link names
M2	Unique NowCerts_Policy_GUID when present; Policy_Number not unique	Duplicate Policy_Number save succeeds
M3	12 verdicts, first-match order in the BRD/PRD	Run Recon on seeds (Phase 2)
M4	Confidence S1-S25 point table, integer 0-100	Seed expected_confidence
M5	Cancel class required on Cancelled / Flat Cancel / Pending Cancel	Save blocked
M6	Is_Rewrite requires Rewrite_Of	Save blocked
M7	Active checkbox true only if Policy_Status = Active	On save
M8	No __c suffixes	Link names
M9	CRM write only if Approved_To_Push AND Approved_By AND Approved_At within 24h AND Recommended_Payload non-empty	Phase 4 stub returns gate failed otherwise
M10	Integration user cannot set Approved_To_Push	Permissions

4. Deliverables by phase

Phase	Deliverable	Issuer action
1	App + 5 forms + fields + picklists + lookups + history hook + views skeleton + permissions	Issuer reviews. Zia waits.

Phase	Deliverable	Issuer action
2	thisapp.recon.* functions, Run Recon button, Policy_Audit / exceptions	Issuer runs sample seeds
3	Schedules 6:45am ET daily, Sunday Tier C, weekday SLA sweep, notifications	Issuer watches one dry run
4	pullCRM read; gated pushCRMIfApproved	Sandbox only
5-6	Optional Hermes health; Analytics	Later RFP addendum

5. Data appendix (must follow)

Artifact	Contents
forms_policy_master.csv (and four sibling CSVs)	Field create list
picklists.csv	Exact option strings
views.csv / workflows.csv	Reports and automation
deluge/01 through 08	Copy-paste functions
tests/sample_records.json	Only allowed seed policies
Process diagrams PDF	Flows Zia must implement

If prose in this RFP and a CSV disagree, **the CSV wins**. If Deluge comments and the scoring table disagree, **the scoring table in the PRD/BRD wins**.

6. Evaluation (how RSG will judge the build)

Weight	Criterion
30%	Schema completeness (five forms, all fields, exact picklists)
25%	Verdict order and seed accuracy (Phase 2)
20%	Guardrails (no AMS write, no Account create, approval gate)
15%	Permissions (Gretchen vs Lamar)
10%	Operational fit (views, SLA hours, stale-touch rule)

Any invented policy number, premium, or GUID in sample data is an automatic fail.

7. Response instructions for Zia

1. Read PRD, BRD, this RFP, and the process diagrams.
2. Load field dictionaries from the attached Excel/CSV/JSON pack.

3. Create the Creator application.
4. Execute **Phase 1 only**.
5. Reply with: form names created, field counts per form, picklist keys loaded, and a list of anything you could not create (field name + reason). Do not silently skip.
6. Do not start Phase 2 until the issuer says to proceed.

8. Legal / operating constraints

- NowCerts is the system of record for policy facts.
- Zoho CRM is the system of record for Accounts, Contacts, Deals.
- Human approval is required for CRM writes. There is no Creator path to AMS writes.
- Test seeds use RSG-TEST-* numbers only.

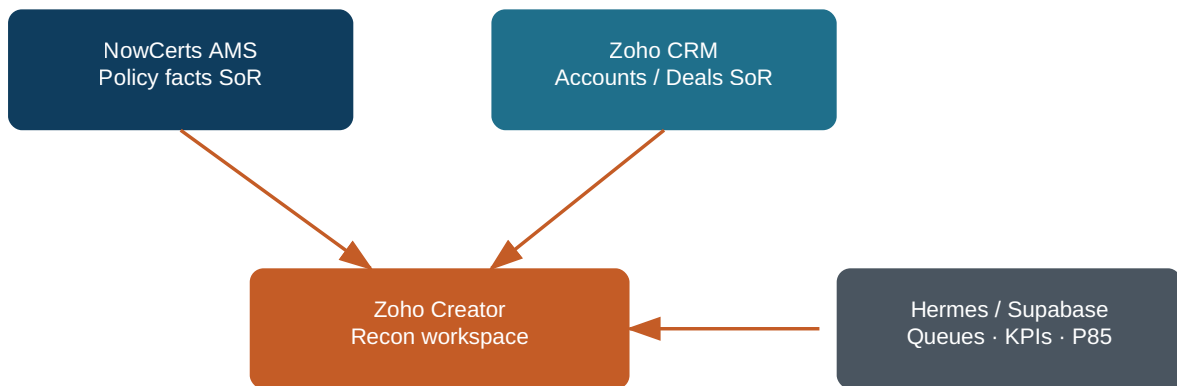
9. Issuer contacts

- Product / approvals: Lamar
- CSR operator: Gretchen
- Build agent: Zia AI on Zoho Creator

Process diagrams

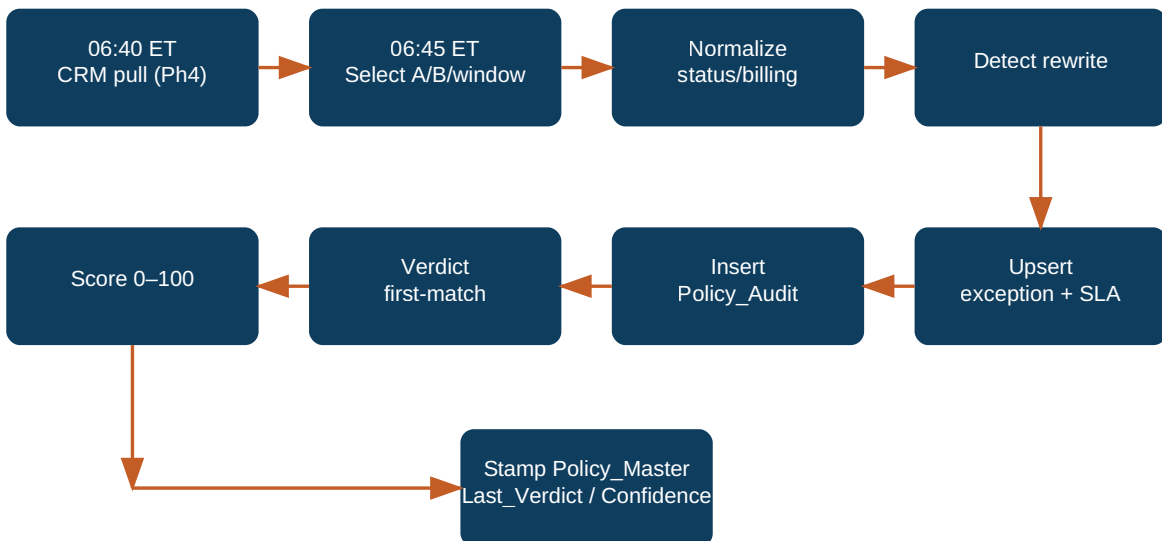
Implement these six flows in Deluge. Do not substitute a different model.

1. Systems of record



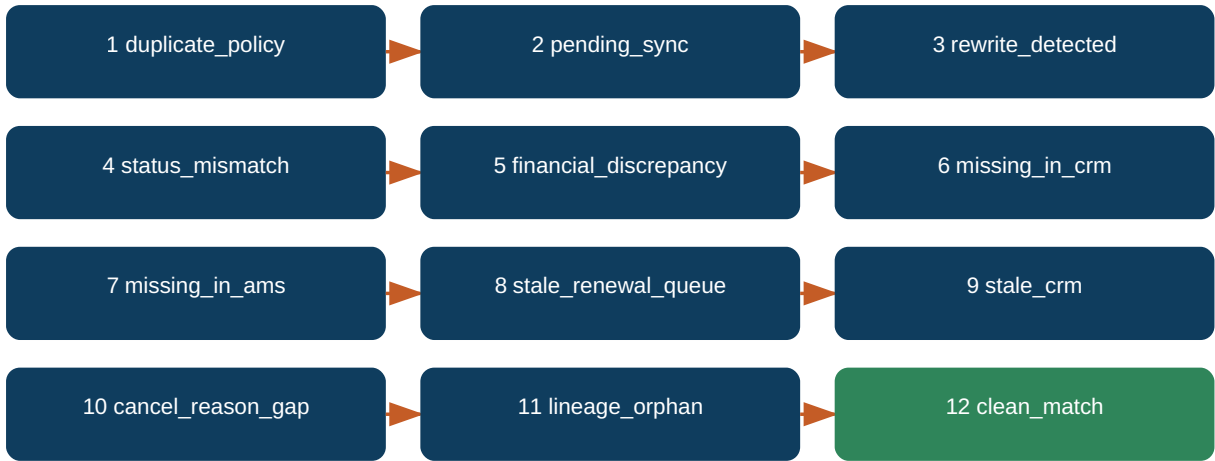
Creator copies AMS facts, stamps CRM IDs, never writes NowCerts, writes CRM only with approval.

2. Daily reconciliation (one Run_ID)

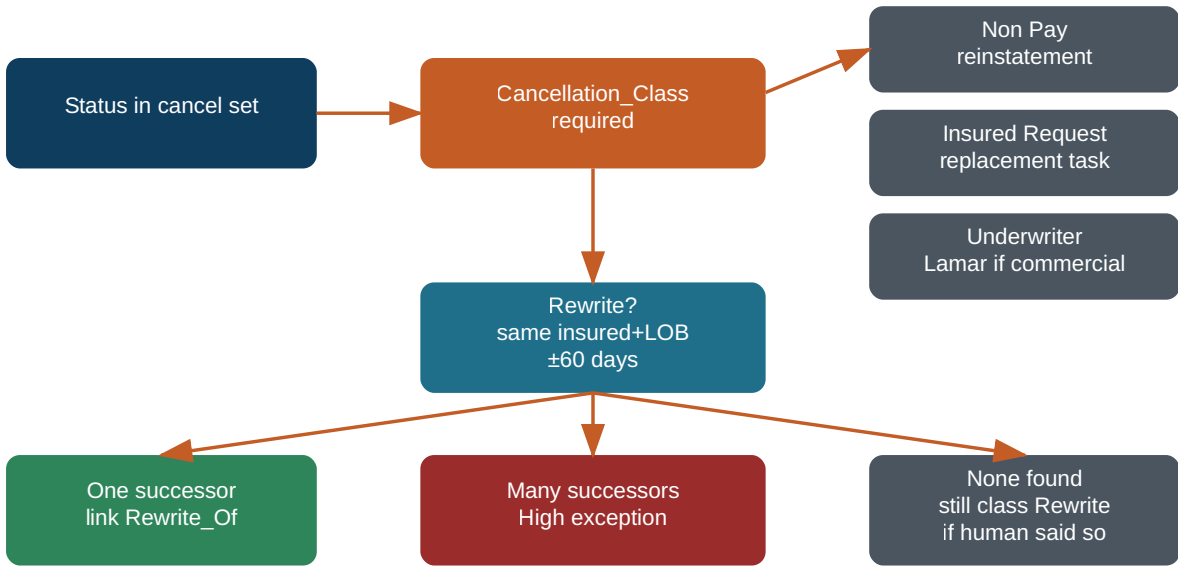


3. Verdict decision tree (order)

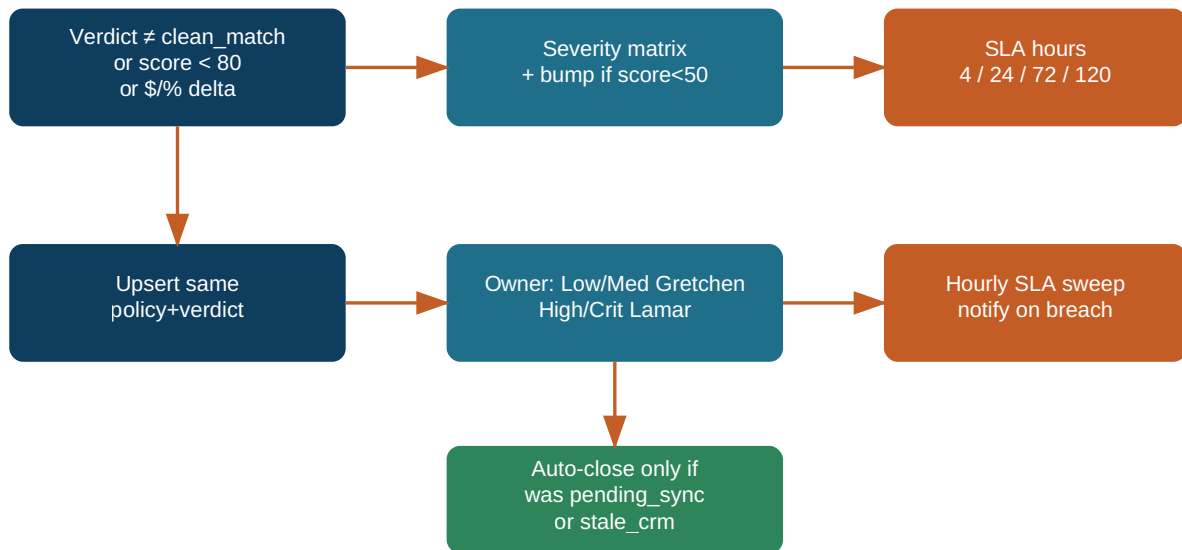
First match wins. Do not add or rename types.



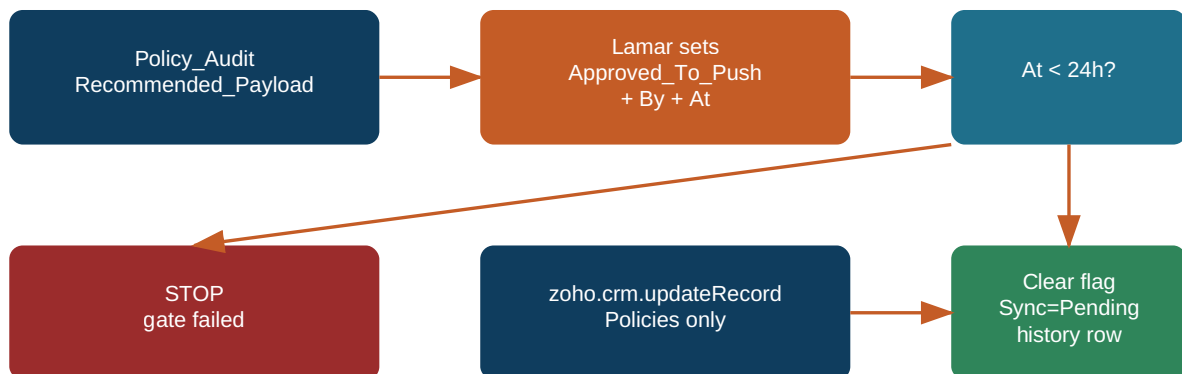
4. Cancellation and rewrite



5. Exception and SLA



6. Approval-gated CRM write



Never create Accounts. Never call NowCerts from Creator. Policy create drafts stay human.

Appendix A — Field dictionaries

Create every row. Use Deluge_Name as the Creator field link name. Do not invent extra fields.

Policy_Master

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory
Policy_Master	A Identity	Policy Name	Name	Single Line	255	Y
Policy_Master	A Identity	Policy Number	Policy_Number	Single Line	100	Y
Policy_Master	A Identity	NowCerts Policy GUID	NowCerts_Policy_GUID	Single Line	36	N
Policy_Master	A Identity	Hermes Policy ID	Hermes_Policy_ID	Single Line	36	N
Policy_Master	A Identity	CRM Policy ID	CRM_Policy_ID	Single Line	50	N
Policy_Master	A Identity	CRM Account ID	CRM_Account_ID	Single Line	50	N
Policy_Master	A Identity	CRM Deal ID	CRM_Deal_ID	Single Line	50	N
Policy_Master	A Identity	NowCerts Insured GUID	NowCerts_Insured_GUID	Single Line	36	N
Policy_Master	B Insured	Insured Name	Insured_Name	Single Line	255	N
Policy_Master	B Insured	Insured Type	Insured_Type	Picklist	50	N
Policy_Master	B Insured	FEIN	FEIN	Single Line	32	N
Policy_Master	B Insured	Email	Email	Email	100	N

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	
Policy_Master	B Insured	Phone	Phone	Phone	30	N	
Policy_Master	B Insured	Billing Street	Billing_Street	Single Line	255	N	
Policy_Master	B Insured	Billing City	Billing_City	Single Line	100	N	
Policy_Master	B Insured	Billing State	Billing_State	Single Line	50	N	
Policy_Master	B Insured	Billing Code	Billing_Code	Single Line	20	N	
Policy_Master	B Insured	Risk State	Risk_State	Single Line	10	N	
Policy_Master	C Coverage	Carrier	Carrier	Single Line	150	N	
Policy_Master	C Coverage	Line of Business	Line_of_Business	Single Line	150	N	
Policy_Master	C Coverage	Business Type	Business_Type	Single Line	100	N	
Policy_Master	C Coverage	Billing Type	Billing_Type	Picklist	40	N	
Policy_Master	C Coverage	Policy Term Months	Policy_Term_Months	Number	3	N	
Policy_Master	C Coverage	Coverage Amount	Coverage_Amount	Currency	16.2	N	
Policy_Master	C Coverage	Deductible	Deductible	Currency	16.2	N	
Policy_Master	D Status	Policy Status	Policy_Status	Picklist	40	N	
Policy_Master	D Status	AMS Status Raw	AMS_Status_Raw	Single Line	80	N	
Policy_Master	D Status	CRM Status	CRM_Status	Picklist	40	N	
Policy_Master	D Status	Active	Active	Checkbox		N	
Policy_Master	D Status	Cancellation Date	Cancellation_Date	Date		N	
Policy_Master	D Status	Cancellation Reason	Cancellation_Reason	Multi Line	2000	N	
Policy_Master	D Status	Cancellation Class	Cancellation_Class	Picklist	40	N	

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	
Policy_Master	D Status	Reinstatement Date	Reinstatement_Date	Date		N	
Policy_Master	E Dates	Effective Date	Effective_Date	Date		N	
Policy_Master	E Dates	Expiration Date	Expiration_Date	Date		N	
Policy_Master	E Dates	Bind Date	Bind_Date	Date		N	
Policy_Master	E Dates	Last AMS Modified	Last_AMS_Modified	Date-Time		N	
Policy_Master	E Dates	Last CRM Modified	Last_CRM_Modified	Date-Time		N	
Policy_Master	E Dates	Last Synced	Last_Synced	Date-Time		N	
Policy_Master	E Dates	Last Audit Date	Last_Audit_Date	Date-Time		N	
Policy_Master	E Dates	Days To Expiration	Days_To_Expiration	Formula Number			
Policy_Master	F Financials	Premium	Premium	Currency	16.2	N	
Policy_Master	F Financials	Current Term Premium	Current_Term_Premium	Currency	16.2	N	
Policy_Master	F Financials	Annualized Premium	Annualized_Premium	Currency	16.2	N	
Policy_Master	F Financials	Agency Commission	Agency_Commission	Currency	16.2	N	
Policy_Master	F Financials	Commission Rate	Commission_Rate	Decimal	5.4	N	
Policy_Master	F Financials	Agency Fee	Agency_Fee	Currency	16.2	N	
Policy_Master	F Financials	CRM Premium	CRM_Premium	Currency	16.2	N	
Policy_Master	F Financials	Premium Delta	Premium_Delta	Formula Currency			
Policy_Master	F Financials	Premium Delta Pct	Premium_Delta_Pct	Formula Decimal			
Policy_Master	G Lineage	Renewed Policy GUID	Renewed_Policy_GUID	Single Line	36	N	

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	
Policy_Master	G Lineage	Predecessor Policy Number	Predecessor_Policy_Number	Single Line	100	N	
Policy_Master	G Lineage	Successor Policy Number	Successor_Policy_Number	Single Line	100	N	
Policy_Master	G Lineage	Rewrite Of	Rewrite_Of	Lookup Policy_Master		N	
Policy_Master	G Lineage	Is Rewrite	Is_Rewrite	Checkbox		N	
Policy_Master	G Lineage	Sync Owner	Sync_Owner	Picklist	40	N	
Policy_Master	H Recon	Sync Status	Sync_Status	Picklist	20	N	
Policy_Master	H Recon	Last Verdict	Last_Verdict	Picklist	40	N	
Policy_Master	H Recon	Last Confidence	Last_Confidence	Number	3	N	
Policy_Master	H Recon	Policy Tier	Policy_Tier	Picklist	5	N	
Policy_Master	H Recon	Assigned To	Assigned_To	Users		N	
Policy_Master	H Recon	Notes	Notes	Multi Line	32000	N	
Policy_Master	H Recon	Approved To Push	Approved_To_Push	Checkbox		N	
Policy_Master	H Recon	Approved By	Approved_By	Users		N	
Policy_Master	H Recon	Approved At	Approved_At	Date-Time		N	
Policy_Master	H Recon	Run Recon On Save	Run_Recon_On_Save	Checkbox		N	
Policy_Master	H Recon	Pending Queue Jobs	Pending_Queue_Jobs	Number	5	N	

Policy_Status_History

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	Unique
Policy_Status_History	Core	Name	Name	Single Line	255	Y	N

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	Unique
Policy_Status_History	Core	Policy	Policy	Lookup Policy_Master		Y	N
Policy_Status_History	Core	Policy Number	Policy_Number	Single Line	100	Y	N
Policy_Status_History	Core	Source System	Source_System	Picklist	20	Y	N
Policy_Status_History	Core	Old Status	Old_Status	Picklist	40	N	N
Policy_Status_History	Core	New Status	New_Status	Picklist	40	Y	N
Policy_Status_History	Core	Old Active	Old_Active	Checkbox		N	N
Policy_Status_History	Core	New Active	New_Active	Checkbox		N	N
Policy_Status_History	Core	Changed At	Changed_At	Date-Time		Y	N
Policy_Status_History	Core	Changed By	Changed_By	Single Line	255	N	N
Policy_Status_History	Core	Change Reason	Change_Reason	Multi Line	2000	N	N
Policy_Status_History	Core	Cancellation Class	Cancellation_Class	Picklist	40	N	N
Policy_Status_History	Core	Raw Payload	Raw_Payload	Multi Line	32000	N	N

Renewal_Queue

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	Unique
Renewal_Queue	Core	Name	Name	Single Line	255	Y	N
Renewal_Queue	Core	Policy	Policy	Lookup Policy_Master		Y	N
Renewal_Queue	Core	Policy Number	Policy_Number	Single Line	100	Y	N
Renewal_Queue	Core	Insured Name	Insured_Name	Single Line	255	Y	N
Renewal_Queue	Core	Carrier	Carrier	Single Line	150	N	N

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	Unique
Renewal_Queue	Core	Line of Business	Line_of_Business	Single Line	150	N	N
Renewal_Queue	Dates	Expiration Date	Expiration_Date	Date		Y	N
Renewal_Queue	Dates	Effective Date	Effective_Date	Date		N	N
Renewal_Queue	Money	Premium Current	Premium_Current	Currency	16.2	N	N
Renewal_Queue	Money	Premium Renewal	Premium_Renewal	Currency	16.2	N	N
Renewal_Queue	Money	Increase Percent	Increase_Percent	Formula Decimal			N
Renewal_Queue	Risk	Risk Status	Risk_Status	Picklist	20	Y	N
Renewal_Queue	Risk	Queue Status	Queue_Status	Picklist	30	Y	N
Renewal_Queue	Risk	Cadence Bucket	Cadence_Bucket	Picklist	20	N	N
Renewal_Queue	Work	Last Contact Date	Last_Contact_Date	Date		N	N
Renewal_Queue	Work	Last Touched At	Last_Touched_At	Date-Time		Y	N
Renewal_Queue	Work	Stale Flag	Stale_Flag	Checkbox		N	N
Renewal_Queue	Work	Stale Reason	Stale_Reason	Multi Line	2000	N	N
Renewal_Queue	Work	Eligibility	Eligibility	Picklist	40	N	N
Renewal_Queue	Work	Assigned To	Assigned_To	Users		N	N
Renewal_Queue	Work	Strategy Notes	Strategy_Notes	Multi Line	32000	N	N
Renewal_Queue	Work	Dismissed	Dismissed	Checkbox		N	N
Renewal_Queue	Work	Hermes Renewal ID	Hermes_Renewal_ID	Single Line	36	N	N

Policy_Audit

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory
Policy_Audit	Core	Name	Name	Single Line	255	Y
Policy_Audit	Core	Policy	Policy	Lookup Policy_Master		Y
Policy_Audit	Core	Policy Number	Policy_Number	Single Line	100	Y
Policy_Audit	Core	Run ID	Run_ID	Single Line	40	Y
Policy_Audit	Core	Run At	Run_At	Date-Time		Y
Policy_Audit	Core	Run Type	Run_Type	Picklist	30	Y
Policy_Audit	Result	Verdict	Verdict	Picklist	40	Y
Policy_Audit	Result	Confidence	Confidence	Number	3	Y
Policy_Audit	Snapshots	AMS Status	AMS_Status	Picklist	40	N
Policy_Audit	Snapshots	CRM Status	CRM_Status	Picklist	40	N
Policy_Audit	Snapshots	Creator Status	Creator_Status	Picklist	40	N
Policy_Audit	Snapshots	AMS Premium	AMS_Premium	Currency	16.2	N
Policy_Audit	Snapshots	CRM Premium	CRM_Premium	Currency	16.2	N
Policy_Audit	Snapshots	Premium Delta	Premium_Delta	Currency	16.2	N
Policy_Audit	Snapshots	Premium Delta Pct	Premium_Delta_Pct	Decimal	8.4	N
Policy_Audit	Flags	Queue Stale	Queue_Stale	Checkbox		N
Policy_Audit	Flags	Duplicate Count	Duplicate_Count	Number	5	N
Policy_Audit	Flags	Rewrite Linked	Rewrite_Linked	Checkbox		N
Policy_Audit	Flags	Pending Queue Jobs	Pending_Queue_Jobs	Number	5	N
Policy_Audit	Text	Findings	Findings	Multi Line	32000	Y
Policy_Audit	Text	Recommendation	Recommendation	Multi Line	32000	Y
Policy_Audit	Text	Recommended Payload	Recommended_Payload	Multi Line	32000	N
Policy_Audit	Text	Score Breakdown	Score_Breakdown	Multi Line	32000	N

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory
Policy_Audit	Link	Exception Created	Exception_Created	Checkbox		N
Policy_Audit	Link	Related Exception	Related_Exception	Lookup Audit_Exceptions		N

Audit_Exceptions

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	Unique
Audit_Exceptions	Core	Name	Name	Single Line	255	Y	N
Audit_Exceptions	Core	Policy	Policy	Lookup Policy_Master		Y	N
Audit_Exceptions	Core	Policy Number	Policy_Number	Single Line	100	Y	N
Audit_Exceptions	Core	Audit	Audit	Lookup Policy_Audit		Y	N
Audit_Exceptions	Core	Verdict	Verdict	Picklist	40	Y	N
Audit_Exceptions	Work	Severity	Severity	Picklist	20	Y	N
Audit_Exceptions	Work	Status	Status	Picklist	20	Y	N
Audit_Exceptions	SLA	SLA Hours	SLA_Hours	Number	5	Y	N
Audit_Exceptions	SLA	SLA Due At	SLA_Due_At	Date-Time		Y	N
Audit_Exceptions	SLA	SLA Breached	SLA_Breached	Checkbox		N	N
Audit_Exceptions	Work	Owner	Owner	Users		Y	N
Audit_Exceptions	Time	Opened At	Opened_At	Date-Time		Y	N
Audit_Exceptions	Time	First Touched At	First_Touched_At	Date-Time		N	N
Audit_Exceptions	Time	Resolved At	Resolved_At	Date-Time		N	N

Form	Section	Display_Label	Deluge_Name	Data_Type	Length	Mandatory	Unique
Audit_Exceptions	Close	Resolution Notes	Resolution_Notes	Multi Line	32000	N	N
Audit_Exceptions	Close	Resolution Class	Resolution_Class	Picklist	40	N	N
Audit_Exceptions	Notify	Escalated To Lamar	Escalated_To_Lamar	Checkbox		N	N
Audit_Exceptions	Notify	Notify Sent	Notify_Sent	Checkbox		N	N
Audit_Exceptions	Notify	Last Notify At	Last_Notify_At	Date-Time		N	N

Appendix B — Picklists, views, workflows

Picklists (exact strings)

picklist_key	sort_order	value	used_on
insured_type	1	Personal	Policy_Master.Insured_Type
insured_type	2	Commercial	Policy_Master.Insured_Type
billing_type	1	Direct Bill	Policy_Master.Billing_Type
billing_type	2	Agency Bill	Policy_Master.Billing_Type
billing_type	3	Direct Bill 100	Policy_Master.Billing_Type
billing_type	4	Agency Bill 100	Policy_Master.Billing_Type
policy_status	1	Active	Policy_Master.Policy_Status;Policy_Master.CRM_Status;Policy
policy_status	2	Up for Renewal	Policy_Master.Policy_Status
policy_status	3	Renewing	Policy_Master.Policy_Status
policy_status	4	Renewed	Policy_Master.Policy_Status
policy_status	5	Rewritten	Policy_Master.Policy_Status
policy_status	6	Expired	Policy_Master.Policy_Status
policy_status	7	Cancelled	Policy_Master.Policy_Status
policy_status	8	Flat Cancel	Policy_Master.Policy_Status
policy_status	9	Pending Cancel	Policy_Master.Policy_Status
policy_status	10	Non-Renewed	Policy_Master.Policy_Status
policy_status	11	Lapsed	Policy_Master.Policy_Status
cancellation_class	1	Non Pay	Policy_Master.Cancellation_Class;Policy_Status_History.Canc
cancellation_class	2	Rewrite	Policy_Master.Cancellation_Class
cancellation_class	3	Insured Request	Policy_Master.Cancellation_Class
cancellation_class	4	Underwriter	Policy_Master.Cancellation_Class
cancellation_class	5	Other	Policy_Master.Cancellation_Class
sync_owner	1	book_sync	Policy_Master.Sync_Owner
sync_owner	2	rsg-import	Policy_Master.Sync_Owner
sync_owner	3	manual	Policy_Master.Sync_Owner

picklist_key	sort_order	value	used_on
sync_status	1	Synced	Policy_Master.Sync_Status
sync_status	2	Pending	Policy_Master.Sync_Status
sync_status	3	Error	Policy_Master.Sync_Status
sync_status	4	Skipped	Policy_Master.Sync_Status
verdict	1	clean_match	Policy_Master.Last_Verdict;Policy_Audit.Verdict;Audit_Except
verdict	2	stale_renewal_queue	Policy_Master.Last_Verdict
verdict	3	stale_crm	Policy_Master.Last_Verdict
verdict	4	duplicate_policy	Policy_Master.Last_Verdict
verdict	5	pending_sync	Policy_Master.Last_Verdict
verdict	6	rewrite_detected	Policy_Master.Last_Verdict
verdict	7	status_mismatch	Policy_Master.Last_Verdict
verdict	8	financial_discrepancy	Policy_Master.Last_Verdict
verdict	9	missing_in_crm	Policy_Master.Last_Verdict
verdict	10	missing_in_ams	Policy_Master.Last_Verdict
verdict	11	cancel_reason_gap	Policy_Master.Last_Verdict
verdict	12	lineage_orphan	Policy_Master.Last_Verdict
policy_tier	1	A	Policy_Master.Policy_Tier
policy_tier	2	B	Policy_Master.Policy_Tier
policy_tier	3	C	Policy_Master.Policy_Tier
source_system	1	AMS	Policy_Status_History.Source_System
source_system	2	CRM	Policy_Status_History.Source_System
source_system	3	Creator	Policy_Status_History.Source_System
source_system	4	Hermes	Policy_Status_History.Source_System
risk_status	1	SAFE	Renewal_Queue.Risk_Status
risk_status	2	AT_RISK	Renewal_Queue.Risk_Status
risk_status	3	CRITICAL	Renewal_Queue.Risk_Status
risk_status	4	RENEWED	Renewal_Queue.Risk_Status
risk_status	5	LAPSED	Renewal_Queue.Risk_Status
queue_status	1	Open	Renewal_Queue.Queue_Status

picklist_key	sort_order	value	used_on
queue_status	2	In Progress	Renewal_Queue.Queue_Status
queue_status	3	Waiting Client	Renewal_Queue.Queue_Status
queue_status	4	Waiting Carrier	Renewal_Queue.Queue_Status
queue_status	5	Completed	Renewal_Queue.Queue_Status
queue_status	6	Dismissed	Renewal_Queue.Queue_Status
queue_status	7	Stale	Renewal_Queue.Queue_Status
cadence_bucket	1	90	Renewal_Queue.Cadence_Bucket
cadence_bucket	2	60	Renewal_Queue.Cadence_Bucket
cadence_bucket	3	30	Renewal_Queue.Cadence_Bucket
cadence_bucket	4	Past Due	Renewal_Queue.Cadence_Bucket
eligibility	1	eligible	Renewal_Queue.Eligibility
eligibility	2	needs_verification	Renewal_Queue.Eligibility
eligibility	3	excluded	Renewal_Queue.Eligibility
run_type	1	scheduled_daily	Policy_Audit.Run_Type
run_type	2	scheduled_tier	Policy_Audit.Run_Type
run_type	3	manual	Policy_Audit.Run_Type
run_type	4	on_edit	Policy_Audit.Run_Type
severity	1	Low	Audit_Exceptions.Severity
severity	2	Medium	Audit_Exceptions.Severity
severity	3	High	Audit_Exceptions.Severity
severity	4	Critical	Audit_Exceptions.Severity
exception_status	1	Open	Audit_Exceptions.Status
exception_status	2	In Progress	Audit_Exceptions.Status
exception_status	3	Waiting	Audit_Exceptions.Status
exception_status	4	Resolved	Audit_Exceptions.Status
exception_status	5	Won't Fix	Audit_Exceptions.Status
exception_status	6	Duplicate	Audit_Exceptions.Status
resolution_class	1	Corrected CRM	Audit_Exceptions.Resolution_Class
resolution_class	2	Corrected Creator	Audit_Exceptions.Resolution_Class

picklist_key	sort_order	value	used_on
resolution_class	3	Linked rewrite	Audit_Exceptions.Resolution_Class
resolution_class	4	True cancel	Audit_Exceptions.Resolution_Class
resolution_class	5	Duplicate merged	Audit_Exceptions.Resolution_Class
resolution_class	6	Data lag	Audit_Exceptions.Resolution_Class
resolution_class	7	Won't fix	Audit_Exceptions.Resolution_Class

Views

View_Name	Form	Audience	Filter	Sort	Columns
All Policies	Policy_Master	Admin	all	Insured_Name asc	Policy_Number;Insured
Active Book	Policy_Master	Both	Active = true	Annualized_Premium desc	Policy_Number;Insured
Needs Recon	Policy_Master	Both	Last_Verdict != clean_match OR Last_Audit_Date is null	Last_Confidence asc	Policy_Number;Insured
CSR Queue	Audit_Exceptions	Gretchen	Status in (Open	In Progress) AND Severity in (Low	Medium)
Lamar Exceptions	Audit_Exceptions	Lamar	Severity in (High	Critical) AND Status not in (Resolved	Won't Fix)
Stale Renewals	Renewal_Queue	Gretchen	Stale_Flag = true OR Queue_Status = Stale	Expiration_Date asc	Policy_Number;Insured
30-Day Radar	Renewal_Queue	Both	Cadence_Bucket in (30	Past Due) AND Dismissed = false	Risk_Status desc then P
Today's Audits	Policy_Audit	Admin	Run_At is today	Run_At desc	Policy_Number;Verdict;
SLA Breaches	Audit_Exceptions	Lamar	SLA_Breached = true AND Status not in (Resolved	Won't Fix)	SLA_Due_At asc
Status History	Policy_Status_History	Admin	all	Changed_At desc	Policy_Number;Source_
Rewrites	Policy_Master	Both	Is_Rewrite = true OR	Cancellation_Date desc	Policy_Number;Insured

View_Name	Form	Audience	Filter	Sort	Columns
			Policy_Status = Rewritten		
Tier A Daily	Policy_Master	Admin	Policy_Tier = A	Last_Audit_Date asc	Policy_Number;Insured

Workflows

Workflow_Name	Form_or_App	Trigger	Phase	Function	Descr
Compose Policy Name	Policy_Master	On add; On edit	1	inline	Set Na + " — Policy_ empty
Derive Active Flag	Policy_Master	On add; On edit	1	inline	Active Policy_
Require Cancel Class	Policy_Master	On add; On edit	1	inline	If Polic Cancel Pending Cancel → aler
Require Rewrite Link	Policy_Master	On add; On edit	1	inline	If Is_R Rewrit
Set Policy Tier	Policy_Master	On add; On edit	1	inline	A if An >= 500 (Insur and Pr else B expira else C
Status History Hook	Policy_Master	On edit (Policy_Status or Active changed)	1	thisapp.recon.writeStatusHistory	Insert Policy_ never
Initial Status History	Policy_Master	On add	1	thisapp.recon.writeStatusHistory	Insert Old_St
Run Recon On Save	Policy_Master	On add; On edit if Run_Recon_On_Save	2	thisapp.recon.runOne	Then o Run_R
Button Run Recon	Policy_Master	Custom button	2	thisapp.recon.runOne	CSR on same
Compose History Name	Policy_Status_History	On add	1	inline	Name " + Ch

Workflow_Name	Form_or_App	Trigger	Phase	Function	Descr
Block History Edit	Policy_Status_History	On edit	1	inline	CSR: d allow only if prefer
Compose Queue Name	Renewal_Queue	On add; On edit	1	inline	Name x " + E
Touch Stamp	Renewal_Queue	On edit of Queue_Status Last_Contact_Date Strategy_Notes Risk_Status	1	inline	Set La now. D other
One Open Queue Row	Renewal_Queue	On add; On edit	1	inline	If Que In Pro Client Stale o same status
Recompute Cadence	Renewal_Queue	On add; On edit Expiration_Date	2	inline	Set Ca days t
Recompute Risk	Renewal_Queue	On add; On edit Premium_* or Expiration_Date	2	inline	Herm first th
Compose Audit Name	Policy_Audit	On add	2	inline	Name @ " + E
Compose Exception Name	Audit_Exceptions	On add; On edit	2	inline	Name Policy Verdic
Exception First Touch	Audit_Exceptions	On edit Status to In Progress	3	inline	If First set no
Exception Resolve Guard	Audit_Exceptions	On edit Status to Resolved	3	inline	Requir and Re Gretch High/O
Exception SLA Hours	Audit_Exceptions	On add	2	thisapp.recon.upsertException	Set SLA severit Opene
Daily Recon Schedule	App	Schedule 06:45 ET daily	3	thisapp.recon.runDaily	Tier A verdic
	App		3	thisapp.recon.runDaily	

Workflow_Name	Form_or_App	Trigger	Phase	Function	Descr
Weekly Tier C Schedule		Schedule Sunday 07:00 ET			Filter P inactiv
SLA Sweep Schedule	App	Schedule hourly 08:00-18:00 ET weekdays	3	thisapp.recon.upsertException	Set SL
CRM Pull Schedule	App	Schedule 06:40 ET daily	4	thisapp.recon.pullCRM	After e read-c
Button Approve CRM Draft	Policy_Master	Custom button Admin only	4	stub	Sets A only; c until s
Button Push CRM Draft	Policy_Master	Custom button Admin only	4	gated stub	Execut zoho.c only if passes

Appendix C — Deluge (install in filename order)

Paste as custom functions `thisapp.recon.*`. Logic must not change.

01_normalize_status.dg

```
// thisapp.recon.normalizeStatus
// Map any AMS/CRM spelling to the canonical Policy_Status picklist, or empty string.
// Source: packages/rsg-hermes-core/hermes_core/canonical.py _STATUS_NORMALIZE

string thisapp.recon.normalizeStatus(string raw)
{
    key = ifnull(raw, "").trim().toLowerCase();
    if(key == "")
    {
        return "";
    }
    table = Map();
    table.put("active", "Active");
    table.put("in force", "Active");
    table.put("inforce", "Active");
    table.put("bound", "Active");
    table.put("up for renewal", "Up for Renewal");
    table.put("renewal pending", "Up for Renewal");
    table.put("renewing", "Renewing");
    table.put("in renewal", "Renewing");
    table.put("renewal", "Renewing");
    table.put("renewed", "Renewed");
    table.put("rewritten", "Rewritten");
    table.put("expired", "Expired");
    table.put("cancelled", "Cancelled");
    table.put("canceled", "Cancelled");
    table.put("flat cancel", "Flat Cancel");
    table.put("flat-cancel", "Flat Cancel");
    table.put("flat cancelled", "Flat Cancel");
    table.put("pending cancel", "Pending Cancel");
    table.put("pending cancellation", "Pending Cancel");
    table.put("cxl pending", "Pending Cancel");
    table.put("non-renewed", "Non-Renewed");
    table.put("non renewed", "Non-Renewed");
    table.put("nonrenewed", "Non-Renewed");
    table.put("lapsed", "Lapsed");
    if(table.containsKey(key))
    {
        return table.get(key);
    }
    return "";
}

// thisapp.recon.normalizeBilling
// Source: hermes_core/canonical.py _BILLING_NORMALIZE

string thisapp.recon.normalizeBilling(string raw)
{
    text = ifnull(raw, "").trim();
    if(text == "")
    {
        return "";
    }
    key = text.toLowerCase().replaceAll("_", " ").trim();
    table = Map();
    table.put("direct bill", "Direct Bill");
    table.put("direct", "Direct Bill");
}
```

```

        table.put("db", "Direct Bill");
        table.put("direct bill autopay", "Direct Bill");
        table.put("db autopay", "Direct Bill");
        table.put("agency bill", "Agency Bill");
        table.put("agency", "Agency Bill");
        table.put("ab", "Agency Bill");
        table.put("agency bill autopay", "Agency Bill");
        table.put("direct bill 100", "Direct Bill 100");
        table.put("db 100", "Direct Bill 100");
        table.put("db100", "Direct Bill 100");
        table.put("agency bill 100", "Agency Bill 100");
        table.put("ab 100", "Agency Bill 100");
        table.put("ab100", "Agency Bill 100");
        if(table.containsKey(key))
        {
            return table.get(key);
        }
        return text.replaceAll("_", " ");
    }

// True if normalized status is a cancel class used by cancel_reason_gap.
bool thisapp.recon.isCancelStatus(string normalized)
{
    s = ifnull(normalized, "");
    if(s == "Cancelled" || s == "Flat Cancel" || s == "Pending Cancel")
    {
        return true;
    }
    return false;
}

// Active checkbox must follow Policy_Status.
bool thisapp.recon.isActiveStatus(string normalized)
{
    return ifnull(normalized, "") == "Active";
}

// Exclude statuses that must not be Active=true.
bool thisapp.recon.isExcludeStatus(string normalized)
{
    s = ifnull(normalized, "");
    if(s == "Expired" || s == "Cancelled" || s == "Flat Cancel" || s == "Non-Renewed" || s ==
"Lapsed")
    {
        return true;
    }
    return false;
}

// Policy tier: A / B / C. Never invent premium.
string thisapp.recon.computeTier(int policyID)
{
    rec = Policy_Master[ID == policyID];
    if(!rec.count())
    {
        return "C";
    }
    ann = ifnull(rec.Annualized_Premium, ifnull(rec.Premium, 0));
    prem = ifnull(rec.Premium, 0);
    itype = ifnull(rec.Insured_Type, "");
    if(ann >= 5000)
    {
        return "A";
    }
}

```

```

if(itype == "Commercial" && prem >= 2500)
{
    return "A";
}
days = 9999;
if(rec.Expiration_Date != null)
{
    days = rec.Expiration_Date.getDayDiff(zoho.currentdate);
    // getDayDiff is date2 - date1 in some accounts; use daysBetween if needed:
    // days = daysBetween(zoho.currentdate, rec.Expiration_Date);
}
if(rec.Active == true || (days >= 0 && days <= 90))
{
    return "B";
}
return "C";
}

```

02_confidence_score.dg

```

// thisapp.recon.score
// Returns 0-100 integer. Also writes last breakdown into a Map returned via
// thisapp.recon.scoreDetail for Policy_Audit.Score_Breakdown.
// Spec section 7. Do not invent extra deductions.

int thisapp.recon.score(int policyID)
{
    detail = thisapp.recon.scoreDetail(policyID);
    return detail.get("score");
}

map thisapp.recon.scoreDetail(int policyID)
{
    rec = Policy_Master[ID == policyID];
    out = Map();
    lines = List();
    score = 100;
    if(!rec.count())
    {
        out.put("score", 0);
        out.put("breakdown", "S0 -100 policy not found");
        return out;
    }
    status = thisapp.recon.normalizeStatus(ifnull(rec.Policy_Status, rec.AMS_Status_Raw));
    crmStatus = thisapp.recon.normalizeStatus(ifnull(rec.CRM_Status, ""));
    amsStatus = thisapp.recon.normalizeStatus(ifnull(rec.AMS_Status_Raw, rec.Policy_Status));
    dup = thisapp.recon.duplicateCount(policyID);
    prem = rec.Premium;
    crmPrem = rec.CRM_Premium;
    deltaAbs = 0.0;
    deltaPct = 0.0;
    hasDelta = false;
    if(prem != null && crmPrem != null)
    {
        deltaAbs = prem - crmPrem;
        if(deltaAbs < 0)
        {
            deltaAbs = deltaAbs * -1;
        }
        hasDelta = true;
        if(crmPrem != 0)
        {
            deltaPct = (deltaAbs / crmPrem) * 100;
        }
    }
    if(ifnull(rec.Policy_Number, "") == "")
    {
        score = score - 40;
    }
}

```

```

        lines.add("S1 -40 Policy_Number blank");
    }
    if(ifnull(rec.NowCerts_Policy_GUID, "") == "")
    {
        score = score - 25;
        lines.add("S2 -25 NowCerts_Policy_GUID blank");
    }
    if(ifnull(rec.CRM_Policy_ID, "") == "")
    {
        score = score - 10;
        lines.add("S3 -10 CRM_Policy_ID blank");
    }
    if(ifnull(rec.Insured_Name, "") == "")
    {
        score = score - 10;
        lines.add("S4 -10 Insured_Name blank");
    }
    if(ifnull(rec.Carrier, "") == "")
    {
        score = score - 8;
        lines.add("S5 -8 Carrier blank");
    }
    if(ifnull(rec.Line_of_Business, "") == "")
    {
        score = score - 5;
        lines.add("S6 -5 Line_of_Business blank");
    }
    if(rec.Effective_Date == null)
    {
        score = score - 10;
        lines.add("S7 -10 Effective_Date blank");
    }
    if(rec.Expiration_Date == null)
    {
        score = score - 10;
        lines.add("S7 -10 Expiration_Date blank");
    }
    if(status == "")
    {
        score = score - 15;
        lines.add("S8 -15 Policy_Status blank or unnormalized");
    }
    if(amsStatus != "" && crmStatus != "" && amsStatus != crmStatus)
    {
        score = score - 12;
        lines.add("S9 -12 AMS vs CRM status differ");
    }
    if(amsStatus != "" && status != "" && amsStatus != status)
    {
        score = score - 12;
        lines.add("S10 -12 AMS vs Creator status differ");
    }
    if(status == "Active" && prem == null)
    {
        score = score - 15;
        lines.add("S11 -15 Premium blank on Active policy");
    }
    if(hasDelta && deltaAbs >= 25)
    {
        score = score - 10;
        lines.add("S12 -10 abs premium $ delta >= 25");
    }
    if(hasDelta && crmPrem != 0 && deltaPct >= 1)
    {
        score = score - 8;
        lines.add("S13 -8 abs premium % delta >= 1");
    }
    if(hasDelta && deltaAbs >= 500)
    {
        score = score - 15;
        lines.add("S14 -15 abs premium $ delta >= 500 extra");
    }
}

```

```

if(dup > 1)
{
    score = score - 30;
    lines.add("S15 -30 Duplicate_Count > 1");
}
if(thisapp.recon.isCancelStatus(status) && ifnull(rec.Cancellation_Class, "") == "")
{
    score = score - 12;
    lines.add("S16 -12 Cancellation without class");
}
if(rec.Is_Rewrite == true && rec.Rewrite_Of == null)
{
    score = score - 15;
    lines.add("S17 -15 Rewrite detected but Rewrite_Of empty");
}
if(thisapp.recon.lineageBroken(policyID))
{
    score = score - 12;
    lines.add("S18 -12 Lineage pointer set but target missing");
}
if(rec.Last_Synced != null)
{
    ageHours = (zoho.currenttime.toLong() - rec.Last_Synced.toLong()) / 3600000;
    if(ageHours > 48)
    {
        score = score - 8;
        lines.add("S19 -8 Last_Synced older than 48h");
    }
    if(ageHours > 168)
    {
        score = score - 12;
        lines.add("S20 -12 Last_Synced older than 7 days extra");
    }
}
if(thisapp.recon.queueIsStale(policyID))
{
    score = score - 10;
    lines.add("S21 -10 Renewal_Queue stale");
}
if(ifnull(rec.Pending_Queue_Jobs, 0) > 0)
{
    score = score - 8;
    lines.add("S22 -8 Pending_Queue_Jobs > 0");
}
if(rec.Active == true && thisapp.recon.isExcludeStatus(status))
{
    score = score - 20;
    lines.add("S23 -20 Active=true but status in EXCLUDE_STATUSES");
}
if(rec.Active == false && status == "Active")
{
    score = score - 15;
    lines.add("S24 -15 Active=false but status = Active");
}
bt = ifnull(rec.Billing_Type, "");
if(status == "Active" && bt.contains("Agency") && rec.Agency_Fee == null)
{
    score = score - 5;
    lines.add("S25 -5 Agency Bill Active missing Agency_Fee");
}
if(score < 0)
{
    score = 0;
}
if(score > 100)
{
    score = 100;
}
out.put("score", score);
out.put("breakdown", lines.toString("\n"));
out.put("duplicate_count", dup);
out.put("delta_abs", deltaAbs);

```



```

        out.put("delta_pct", deltaPct);
        return out;
    }

    int thisapp.recon.duplicateCount(int policyID)
    {
        rec = Policy_Master[ID == policyID];
        if(!rec.count())
        {
            return 0;
        }
        n = 0;
        num = ifnull(rec.Policy_Number, "");
        guid = ifnull(rec.NowCerts_Policy_GUID, "");
        if(num != "")
        {
            n = Policy_Master[Policy_Number == num].count();
        }
        g = 0;
        if(guid != "")
        {
            g = Policy_Master[NowCerts_Policy_GUID == guid].count();
        }
        if(g > n)
        {
            return g;
        }
        return n;
    }

    bool thisapp.recon.lineageBroken(int policyID)
    {
        rec = Policy_Master[ID == policyID];
        if(!rec.count())
        {
            return false;
        }
        if(ifnull(rec.Renewed_Policy_GUID, "") != "")
        {
            found = Policy_Master[NowCerts_Policy_GUID == rec.Renewed_Policy_GUID].count();
            if(found == 0)
            {
                return true;
            }
        }
        if(ifnull(rec.Successor_Policy_Number, "") != "")
        {
            found2 = Policy_Master[Policy_Number == rec.Successor_Policy_Number].count();
            if(found2 == 0)
            {
                return true;
            }
        }
        return false;
    }

    bool thisapp.recon.queueIsStale(int policyID)
    {
        q = Renewal_Queue[Policy == policyID && Dismissed == false && Queue_Status in {"Open", "In
Progress", "Waiting Client", "Waiting Carrier", "Stale"}];
        if(!q.count())
        {
            return false;
        }
        row = q.getAll().get(0);
        stale = thisapp.recon.evaluateQueueStale(row.get("ID"));
        return stale.get("stale");
    }

```

03_verdict_engine.dg

```
// thisapp.recon.verdict
// First-match-wins matrix. Spec section 6.1. Do not add verdict types.

string thisapp.recon.verdict(int policyID)
{
    detail = thisapp.recon.verdictDetail(policyID);
    return detail.get("verdict");
}

map thisapp.recon.verdictDetail(int policyID)
{
    rec = Policy_Master[ID == policyID];
    out = Map();
    findings = List();
    if(!rec.count())
    {
        out.put("verdict", "missing_in_ams");
        out.put("findings", "Policy_Master row not found");
        out.put("recommendation", "Do not invent a policy. Re-run after a valid ID.");
        return out;
    }
    status = thisapp.recon.normalizeStatus(ifnull(rec.Policy_Status, rec.AMS_Status_Raw));
    crmStatus = thisapp.recon.normalizeStatus(ifnull(rec.CRM_Status, ""));
    amsStatus = thisapp.recon.normalizeStatus(ifnull(rec.AMS_Status_Raw, rec.Policy_Status));
    dup = thisapp.recon.duplicateCount(policyID);
    prem = rec.Premium;
    crmPrem = rec.CRM_Premium;
    deltaAbs = 0.0;
    deltaPct = 0.0;
    hasBothPrem = false;
    if(prem != null && crmPrem != null)
    {
        hasBothPrem = true;
        deltaAbs = prem - crmPrem;
        if(deltaAbs < 0)
        {
            deltaAbs = deltaAbs * -1;
        }
        if(crmPrem != 0)
        {
            deltaPct = (deltaAbs / crmPrem) * 100;
        }
    }
    rewriteHeuristic = thisapp.recon.detectRewrite(policyID);

    // 1 duplicate_policy
    if(dup > 1)
    {
        findings.add("Duplicate_Count=" + dup + " for Policy_Number or NowCerts_Policy_GUID.");
        return thisapp.recon._pack("duplicate_policy", findings);
    }

    // 2 pending_sync
    if(ifnull(rec.Sync_Status, "") == "Pending" || ifnull(rec.Pending_Queue_Jobs, 0) > 0)
    {
        findings.add("Sync_Status=" + ifnull(rec.Sync_Status, "") + " Pending_Queue_Jobs=" +
ifnull(rec.Pending_Queue_Jobs, 0));
        return thisapp.recon._pack("pending_sync", findings);
    }

    // 3 rewrite_detected
    if(rec.Is_Rewrite == true || ifnull(rec.Cancellation_Class, "") == "Rewrite" ||
rewriteHeuristic.get("detected") == true)
    {
        findings.add("Is_Rewrite=" + rec.Is_Rewrite + " class=" + ifnull(rec.Cancellation_Class,
"") + " heuristic=" + rewriteHeuristic.get("reason"));
        return thisapp.recon._pack("rewrite_detected", findings);
    }
}
```

```

    }

    // 4 status_mismatch
    mismatch = false;
    if(amsStatus != "" && crmStatus != "" && amsStatus != crmStatus)
    {
        mismatch = true;
        findings.add("AMS status " + amsStatus + " != CRM status " + crmStatus);
    }
    if(crmStatus != "" && status != "" && crmStatus != status)
    {
        mismatch = true;
        findings.add("CRM status " + crmStatus + " != Creator status " + status);
    }
    if(mismatch)
    {
        return thisapp.recon._pack("status_mismatch", findings);
    }

    // 5 financial_discrepancy (hybrid $25 or 1%)
    if(hasBothPrem && crmPrem != 0 && (deltaAbs >= 25 || deltaPct >= 1))
    {
        findings.add("Premium AMS=" + prem + " CRM=" + crmPrem + " abs$=" + deltaAbs + " abs%=" +
deltaPct);
        return thisapp.recon._pack("financial_discrepancy", findings);
    }
    if(hasBothPrem && crmPrem == 0 && prem != 0 && deltaAbs >= 25)
    {
        findings.add("CRM premium 0; AMS premium " + prem + " abs$=" + deltaAbs);
        return thisapp.recon._pack("financial_discrepancy", findings);
    }

    // 6 missing_in_crm
    syncedAgeOk = false;
    if(rec.Last_Synced != null)
    {
        ageHours = (zoho.currenttime.toLong() - rec.Last_Synced.toLong()) / 3600000;
        if(ageHours > 24)
        {
            syncedAgeOk = true;
        }
    }
    else
    {
        syncedAgeOk = true;
    }
    if(ifnull(rec.NowCerts_Policy_GUID, "") != "" && ifnull(rec.CRM_Policy_ID, "") == "" &&
syncedAgeOk)
    {
        findings.add("AMS GUID present; CRM_Policy_ID empty; Last_Synced older than 24h or
blank.");
        return thisapp.recon._pack("missing_in_crm", findings);
    }

    // 7 missing_in_ams
    if(ifnull(rec.CRM_Policy_ID, "") != "" && ifnull(rec.NowCerts_Policy_GUID, "") == "")
    {
        findings.add("CRM_Policy_ID present; NowCerts_Policy_GUID empty. Do not tombstone.");
        return thisapp.recon._pack("missing_in_ams", findings);
    }

    // 8 stale_renewal_queue
    qstale = thisapp.recon.evaluateQueueStaleFromPolicy(policyID);
    if(qstale.get("stale") == true)
    {
        findings.add(qstale.get("reason"));
        return thisapp.recon._pack("stale_renewal_queue", findings);
    }

    // 9 stale_crm - v1: CRM clock more than 1 hour behind AMS is enough
    // when status/premium already matched (else a higher verdict fired).
    if(rec.Last_AMS_Modified != null && rec.Last_CRM_Modified != null)

```

```

    {
        if(rec.Last_CRM_Modified < rec.Last_AMS_Modified.addHour(-1))
        {
            findings.add("Last_CRM_Modified < Last_AMS_Modified - 1h. Trust AMS; draft CRM
refresh.");
            return thisapp.recon._pack("stale_crm", findings);
        }
    }

    // 10 cancel_reason_gap
    if(thisapp.recon.isCancelStatus(status) && ifnull(rec.Cancellation_Class, "") == "")
    {
        findings.add("Cancelled-class status without Cancellation_Class.");
        return thisapp.recon._pack("cancel_reason_gap", findings);
    }

    // 11 lineage_orphan
    if(thisapp.recon.lineageBroken(policyID))
    {
        findings.add("Renewed_Policy_GUID or Successor_Policy_Number does not resolve.");
        return thisapp.recon._pack("lineage_orphan", findings);
    }

    // 12 clean_match
    findings.add("AMS, CRM, Creator, and renewal queue checks passed.");
    return thisapp.recon._pack("clean_match", findings);
}

map thisapp.recon._pack(string verdict, list findings)
{
    out = Map();
    out.put("verdict", verdict);
    out.put("findings", findings.toString("\n"));
    out.put("recommendation", thisapp.recon.recommendation(verdict));
    return out;
}

string thisapp.recon.recommendation(string verdict)
{
    table = Map();
    table.put("duplicate_policy", "Merge duplicate Policy_Master rows; keep the row with
NowCerts_Policy_GUID + latest Last_AMS_Modified. Do not delete until Lamar confirms.");
    table.put("pending_sync", "Do not edit AMS-owned fields. Wait for sync or inspect outbound
queue.");
    table.put("rewrite_detected", "Link Rewrite_Of / Successor. Move old Renewal_Queue to Completed or
Dismissed. Open replacement coverage task if insured still needs a policy.");
    table.put("status_mismatch", "Trust AMS status. Draft CRM Policy_Status update. Do not execute
without approval.");
    table.put("financial_discrepancy", "Trust AMS premium. Draft CRM premium update. If delta > $500
or Policy_Tier A, assign Lamar.");
    table.put("missing_in_crm", "Draft CRM Policy create from Policy_Master. Link Account by
NowCerts_Insured_GUID. Never create a new Account from this agent.");
    table.put("missing_in_ams", "Do not tombstone. Flag for human: confirm live NowCerts. Historical
false tombstones existed in the book mirror.");
    table.put("stale_renewal_queue", "Assign Gretchen; Cadence_Bucket 30/Past Due is same-day. Update
Last_Touched_At after real contact, not after opening the record.");
    table.put("stale_crm", "Draft CRM field refresh from AMS snapshot on Policy_Master.");
    table.put("cancel_reason_gap", "Gretchen classifies Cancellation_Class: Non Pay, Rewrite, Insured
Request, Underwriter, or Other.");
    table.put("lineage_orphan", "Search Policy_Number / GUID; link or clear the broken pointer. Do not
invent a GUID.");
    table.put("clean_match", "No action. Stamp Last_Audit_Date.");
    if(table.containsKey(verdict))
    {
        return table.get(verdict);
    }
    return "Unknown verdict – stop. Do not invent a handling path.";
}

```

```

string thisapp.recon.severityFor(string verdict, int policyID, int confidence)
{
    rec = Policy_Master[ID == policyID];
    sev = "Medium";
    if(verdict == "duplicate_policy")
    {
        sev = "High";
    }
    else if(verdict == "pending_sync")
    {
        sev = "Low";
    }
    else if(verdict == "rewrite_detected")
    {
        sev = "Medium";
    }
    else if(verdict == "status_mismatch")
    {
        sev = "Medium";
        amsStatus = thisapp.recon.normalizeStatus(ifnull(rec.AMS_Status_Raw, rec.Policy_Status));
        crmStatus = thisapp.recon.normalizeStatus(ifnull(rec.CRM_Status, ""));
        if((amsStatus == "Active" && thisapp.recon.isCancelStatus(crmStatus)) || (crmStatus ==
"Active" && thisapp.recon.isCancelStatus(amsStatus)))
        {
            sev = "High";
        }
    }
    else if(verdict == "financial_discrepancy")
    {
        sev = "Medium";
        if(rec.Premium != null && rec.CRM_Premium != null)
        {
            d = rec.Premium - rec.CRM_Premium;
            if(d < 0)
            {
                d = d * -1;
            }
            if(d >= 500 || rec.Policy_Tier == "A")
            {
                sev = "High";
            }
        }
    }
    else if(verdict == "missing_in_crm")
    {
        sev = "Medium";
        if(rec.Policy_Tier == "A" || ifnull(rec.Premium, 0) >= 5000)
        {
            sev = "High";
        }
    }
    else if(verdict == "missing_in_ams")
    {
        sev = "High";
    }
    else if(verdict == "stale_renewal_queue")
    {
        sev = "Medium";
        q = Renewal_Queue[Policy == policyID && Dismissed == false];
        if(q.count())
        {
            row = q.getAll().get(0);
            if(row.get("Cadence_Bucket") == "Past Due" || row.get("Risk_Status") ==
"CRITICAL")
            {
                sev = "Critical";
            }
        }
    }
    else if(verdict == "stale_crm")
    {
        sev = "Low";
    }
}

```

```

    }
    else if(verdict == "cancel_reason_gap")
    {
        sev = "Medium";
    }
    else if(verdict == "lineage_orphan")
    {
        sev = "Medium";
    }
    else if(verdict == "clean_match")
    {
        return "";
    }
    if(confidence < 50)
    {
        if(sev == "Low")
        {
            sev = "Medium";
        }
        else if(sev == "Medium")
        {
            sev = "High";
        }
        else if(sev == "High")
        {
            sev = "Critical";
        }
    }
    return sev;
}

```

04_rewrite_detection.dg

```

// thisapp.recon.detectRewrite
// Heuristic spec section 8.2. Never invent a successor policy.

map thisapp.recon.detectRewrite(int policyID)
{
    rec = Policy_Master[ID == policyID];
    out = Map();
    out.put("detected", false);
    out.put("reason", "");
    out.put("successor_id", 0);
    if(!rec.count())
    {
        out.put("reason", "policy not found");
        return out;
    }
    status = thisapp.recon.normalizeStatus(ifnull(rec.Policy_Status, rec.AMS_Status_Raw));
    cancelLike = thisapp.recon.isCancelStatus(status) || status == "Non-Renewed" || status ==
"Rewritten";
    if(!cancelLike && ifnull(rec.Cancellation_Class, "") != "Rewrite")
    {
        out.put("reason", "not a cancel/rewrite candidate");
        return out;
    }
    insured = ifnull(rec.NowCerts_Insured_GUID, "");
    lob = ifnull(rec.Line_of_Business, "");
    if(insured == "" || lob == "")
    {
        out.put("reason", "missing NowCerts_Insured_GUID or Line_of_Business – cannot heuristic-
match");
        return out;
    }
    anchor = rec.Cancellation_Date;
    if(anchor == null)
    {
        anchor = rec.Expiration_Date;
    }
}

```

```

        if(anchor == null)
        {
            out.put("reason", "no Cancellation_Date or Expiration_Date");
            return out;
        }
        start = anchor.addDay(-60);
        end = anchor.addDay(60);
        candS = Policy_Master[NowCerts_Insured_GUID == insured && Line_of_Business == lob && ID !=
policyID && Effective_Date >= start && Effective_Date <= end];
        ids = List();
        for each row in candS
        {
            st = thisapp.recon.normalizeStatus(ifnull(row.Policy_Status, row.AMS_Status_Raw));
            if(st == "Active" || st == "Up for Renewal" || st == "Renewing")
            {
                if(ifnull(row.NowCerts_Policy_GUID, "") != ifnull(rec.NowCerts_Policy_GUID, ""))
                {
                    ids.add(row.ID);
                }
            }
        }
        n = ids.size();
        if(n == 0)
        {
            if(ifnull(rec.Cancellation_Class, "") == "Rewrite" || rec.Is_Rewrite == true)
            {
                out.put("detected", true);
                out.put("reason", "classed as rewrite but no successor found in 60-day window");
            }
            else
            {
                out.put("reason", "no successor in 60-day window");
            }
            return out;
        }
        if(n > 1)
        {
            out.put("detected", true);
            out.put("reason", "multiple successor candidates (" + n + "); human must pick");
            out.put("successor_id", 0);
            return out;
        }
        sid = ids.get(0);
        succ = Policy_Master[ID == sid];
        out.put("detected", true);
        out.put("reason", "linked successor " + succ.Policy_Number);
        out.put("successor_id", sid);
        return out;
    }
}

```

// Apply rewrite links when exactly one successor is found.
// Does not change AMS. Updates Creator lineage fields only.

```

void thisapp.recon.applyRewriteLink(int policyID)
{
    info = thisapp.recon.detectRewrite(policyID);
    if(info.get("detected") != true)
    {
        return;
    }
    sid = info.get("successor_id");
    if(sid == 0)
    {
        old = Policy_Master[ID == policyID];
        old.Is_Rewrite=true;
        old.Last_Verdict="rewrite_detected";
        return;
    }
    old = Policy_Master[ID == policyID];
    succ = Policy_Master[ID == sid];
    old.Is_Rewrite=true;
}

```

```

        old.Successor_Policy_Number=succ.Policy_Number;
        if(ifnull(old.Cancellation_Class, "") == "Rewrite" &&
thisapp.recon.normalizeStatus(old.Policy_Status) == "Cancelled")
        {
            old.Policy_Status="Rewritten";
            old.Active=false;
        }
        succ.Rewrite_Of=old.ID;
        q = Renewal_Queue[Policy == policyID && Dismissed == false && Queue_Status in {"Open", "In
Progress", "Waiting Client", "Waiting Carrier", "Stale"}];
        for each qr in q
        {
            qr.Queue_Status="Dismissed";
            qr.Dismissed=true;
            qr.Stale_Reason="rewrite_detected";
            qr.Last_Touched_At=zoho.currenttime;
        }
    }
}

```

05_status_history_hook.dg

```

// thisapp.recon.writeStatusHistory
// Called from Policy_Master on add / on edit when Policy_Status or Active changes.

int thisapp.recon.writeStatusHistory(int policyID, string oldStatus, string newStatus, string source)
{
    rec = Policy_Master[ID == policyID];
    if(!rec.count())
    {
        return 0;
    }
    oldA = false;
    newA = rec.Active;
    if(thisapp.recon.isActiveStatus(oldStatus))
    {
        oldA = true;
    }
    stamp = zoho.currenttime;
    nm = rec.Policy_Number + " " + stamp;
    row = insert into Policy_Status_History
    [
        Name=nm
        Policy=policyID
        Policy_Number=rec.Policy_Number
        Source_System=ifnull(source, "Creator")
        Old_Status=oldStatus
        New_Status=newStatus
        Old_Active=oldA
        New_Active=newA
        Changed_At=stamp
        Changed_By=zoho.loginuser
        Cancellation_Class=rec.Cancellation_Class
    ];
    return row;
}

// Form workflow snippet (On Edit, Policy_Master):
//
// oldS = old.Policy_Status;
// newS = input.Policy_Status;
// if(oldS != newS || old.Active != input.Active)
// {
//     thisapp.recon.writeStatusHistory(input.ID, oldS, newS, "Creator");
// }

```


06_exception_sla.dg

```
// Exception upsert + SLA. Spec sections 6.3, 9.3, 11.

int thisapp.recon.slaHours(string severity)
{
    if(severity == "Critical")
    {
        return 4;
    }
    if(severity == "High")
    {
        return 24;
    }
    if(severity == "Medium")
    {
        return 72;
    }
    return 120;
}

int thisapp.recon.upsertException(int auditID)
{
    aud = Policy_Audit[ID == auditID];
    if(!aud.count())
    {
        return 0;
    }
    verdict = aud.Verdict;
    if(verdict == "clean_match")
    {
        thisapp.recon.maybeAutoClose(aud.Policy, verdict);
        return 0;
    }
    conf = ifnull(aud.Confidence, 0);
    sev = thisapp.recon.severityFor(verdict, aud.Policy, conf);
    hours = thisapp.recon.slaHours(sev);
    opened = zoho.currenttime;
    due = opened.addHour(hours);
    existing = Audit_Exceptions[Policy == aud.Policy && Verdict == verdict && Status in {"Open", "In
Progress", "Waiting"}];
    nm = sev + " - " + aud.Policy_Number + " - " + verdict;
    if(existing.count())
    {
        ex = existing;
        ex.Audit=auditID;
        ex.Severity=sev;
        ex.Name=nm;
        if(thisapp.recon.severityRank(sev) > thisapp.recon.severityRank(ex.Severity))
        {
            ex.SLA_Hours=hours;
            ex.SLA_Due_At=due;
        }
        aud.Exception_Created=true;
        aud.Related_Exception=ex.ID;
        return ex.ID;
    }
    ownerEmail = thisapp.recon.defaultOwner(sev);
    row = insert into Audit_Exceptions
    [
        Name=nm
        Policy=aud.Policy
        Policy_Number=aud.Policy_Number
        Audit=auditID
        Verdict=verdict
        Severity=sev
        Status="Open"
        SLA_Hours=hours
        SLA_Due_At=due
    ]
}
```

```

        SLA_Breached=false
        Owner=ownerEmail
        Opened_At=opened
        Notify_Sent=false
    };
    aud.Exception_Created=true;
    aud.Related_Exception=row;
    return row;
}

int thisapp.recon.severityRank(string sev)
{
    if(sev == "Critical")
    {
        return 4;
    }
    if(sev == "High")
    {
        return 3;
    }
    if(sev == "Medium")
    {
        return 2;
    }
    return 1;
}

// Assign by role label. Replace with actual Creator user emails in production.
string thisapp.recon.defaultOwner(string severity)
{
    if(severity == "High" || severity == "Critical")
    {
        return "lamar";
    }
    return "gretchen";
}

void thisapp.recon.maybeAutoClose(int policyID, string newVerdict)
{
    if(newVerdict != "clean_match")
    {
        return;
    }
    openRows = Audit_Exceptions[Policy == policyID && Status in {"Open", "In Progress", "Waiting"}];
    for each ex in openRows
    {
        if(ex.Verdict == "pending_sync" || ex.Verdict == "stale_crm")
        {
            ex.Status="Resolved";
            ex.Resolution_Class="Data lag";
            ex.Resolution_Notes="Auto-closed: subsequent audit was clean_match.";
            ex.Resolved_At=zoho.currenttime;
        }
    }
}

// Hourly SLA sweep.
void thisapp.recon.slaSweep()
{
    openRows = Audit_Exceptions[Status in {"Open", "In Progress", "Waiting"} && SLA_Breached ==
false];
    for each ex in openRows
    {
        if(ex.SLA_Due_At != null && zoho.currenttime > ex.SLA_Due_At)
        {
            ex.SLA_Breached=true;

```

```

        thisapp.recon.notifyException(ex.ID, "SLA_Breached");
    }
}

void thisapp.recon.notifyException(int exceptionID, string eventName)
{
    ex = Audit_Exceptions[ID == exceptionID];
    if(!ex.count())
    {
        return;
    }
    if(eventName != "SLA_Breached")
    {
        if(ex.Notify_Sent == true && ex.Last_Notify_At != null)
        {
            ageHours = (zoho.currenttime.toLong() - ex.Last_Notify_At.toLong()) / 3600000;
            if(ex.Severity == "Critical" && ageHours < 24)
            {
                return;
            }
        }
        subj = "[RSG Recon] " + eventName + " " + ex.Severity + " " + ex.Policy_Number + " " + ex.Verdict;
        body = "Policy: " + ex.Policy_Number + "\nVerdict: " + ex.Verdict + "\nSeverity: " + ex.Severity +
"\nDue: " + ex.SLA_Due_At;
        // Replace with actual user emails after install.
        sendmail
        [
            from: zoho.adminuserid
            to: zoho.adminuserid
            subject: subj
            message: body
        ]
        ex.Notify_Sent=true;
        ex.Last_Notify_At=zoho.currenttime;
        if(ex.Severity == "High" || ex.Severity == "Critical")
        {
            ex.Escalated_To_Lamar=true;
        }
    }
}

```

07_scheduled_daily_recon.dg

```

// Orchestrator: one policy + daily batch. Spec section 9.

int thisapp.recon.runOne(int policyID, string runType, string runID)
{
    rec = Policy_Master[ID == policyID];
    if(!rec.count())
    {
        return 0;
    }
    if(ifnull(runID, "") == "")
    {
        runID = zoho.currenttime.toString("yyyyMMddHHmmss");
    }
    if(ifnull(runType, "") == "")
    {
        runType = "manual";
    }
    norm = thisapp.recon.normalizeStatus(ifnull(rec.AMS_Status_Raw, rec.Policy_Status));
    if(norm != "" && rec.Policy_Status != norm)
    {
        rec.Policy_Status=norm;
        rec.Active=thisapp.recon.isActiveStatus(norm);
    }
    bt = thisapp.recon.normalizeBilling(ifnull(rec.Billing_Type, ""));
}

```

```

if(bt != "")
{
    rec.Billing_Type=bt;
}
rec.Policy_Tier=thisapp.recon.computeTier(policyID);
rw = thisapp.recon.detectRewrite(policyID);
if(rw.get("detected") == true && rw.get("successor_id") != 0)
{
    thisapp.recon.applyRewriteLink(policyID);
}
thisapp.recon.refreshQueueDerived(policyID);
scoreMap = thisapp.recon.scoreDetail(policyID);
vmap = thisapp.recon.verdictDetail(policyID);
verdict = vmap.get("verdict");
score = scoreMap.get("score");
stamp = zoho.currenttime;
qstale = thisapp.recon.evaluateQueueStaleFromPolicy(policyID);
prem = rec.Premium;
crmPrem = rec.CRM_Premium;
delta = null;
deltaPct = null;
if(prem != null && crmPrem != null)
{
    delta = prem - crmPrem;
    if(crmPrem != 0)
    {
        dabs = delta;
        if(dabs < 0)
        {
            dabs = dabs * -1;
        }
        deltaPct = (dabs / crmPrem) * 100;
    }
}
nm = rec.Policy_Number + " @ " + stamp;
aud = insert into Policy_Audit
[
    Name=nm
    Policy=policyID
    Policy_Number=rec.Policy_Number
    Run_ID=runID
    Run_At=stamp
    Run_Type=runType
    Verdict=verdict
    Confidence=score
    AMS_Status=thisapp.recon.normalizeStatus(ifnull(rec.AMS_Status_Raw, rec.Policy_Status))
    CRM_Status=thisapp.recon.normalizeStatus(ifnull(rec.CRM_Status, ""))
    Creator_Status=thisapp.recon.normalizeStatus(ifnull(rec.Policy_Status, ""))
    AMS_Premium=prem
    CRM_Premium=crmPrem
    Premium_Delta=delta
    Premium_Delta_Pct=deltaPct
    Queue_Stale=qstale.get("stale")
    Duplicate_Count=scoreMap.get("duplicate_count")
    Rewrite_Linked=if(rec.Rewrite_Of != null, true, false)
    Pending_Queue_Jobs=ifnull(rec.Pending_Queue_Jobs, 0)
    Findings=vmap.get("findings")
    Recommendation=vmap.get("recommendation")
    Score_Breakdown=scoreMap.get("breakdown")
    Exception_Created=false
];
rec.Last_Verdict=verdict;
rec.Last_Confidence=score;
rec.Last_Audit_Date=stamp;
rec.Run_Recon_On_Save=false;
thisapp.recon.upsertException(aud);
if(verdict != "clean_match")
{
    ex = Audit_Exceptions[Audit == aud && Status == "Open"];
    if(ex.count() && (ex.Severity == "Critical" || (ex.Severity == "High" && verdict ==
"missing_in_ams"))))
    {

```

```

        thisapp.recon.notifyException(ex.ID, "New exception");
    }
    }
    return aud;
}

void thisapp.recon.runDaily()
{
    runID = "daily-" + zoho.currenttime.toString("yyyyMMddHHmmss");
    windowDate = zoho.currentdate.addDay(120);
    targets = Policy_Master[Policy_Tier == "A" || Policy_Tier == "B" || Active == true ||
Expiration_Date <= windowDate || Last_Verdict != "clean_match" || Last_Audit_Date == null];
    for each rec in targets
    {
        thisapp.recon.runOne(rec.ID, "scheduled_daily", runID);
    }
    thisapp.recon.slaSweep();
}

void thisapp.recon.runWeeklyTierC()
{
    runID = "tierc-" + zoho.currenttime.toString("yyyyMMddHHmmss");
    targets = Policy_Master[Policy_Tier == "C"];
    for each rec in targets
    {
        thisapp.recon.runOne(rec.ID, "scheduled_tier", runID);
    }
}

void thisapp.recon.refreshQueueDerived(int policyID)
{
    q = Renewal_Queue[Policy == policyID && Dismissed == false];
    if(!q.count())
    {
        return;
    }
    for each row in q
    {
        row.Cadence_Bucket=thisapp.recon.cadenceBucket(row.Expiration_Date);
        row.Risk_Status=thisapp.recon.classifyRisk(row.ID);
        st = thisapp.recon.evaluateQueueStale(row.ID);
        row.Stale_Flag=st.get("stale");
        row.Stale_Reason=st.get("reason");
        if(st.get("stale") == true && row.Queue_Status == "Open")
        {
            row.Queue_Status="Stale";
        }
    }
}

string thisapp.recon.cadenceBucket(date exp)
{
    if(exp == null)
    {
        return "";
    }
    days = daysBetween(zoho.currentdate, exp);
    if(days < 0)
    {
        return "Past Due";
    }
    if(days <= 30)
    {
        return "30";
    }
    if(days <= 60)
    {
        return "60";
    }
}

```

```

    }
    if(days <= 90)
    {
        return "90";
    }
    return "";
}

```

```

string thisapp.recon.classifyRisk(int queueID)
{
    row = Renewal_Queue[ID == queueID];
    if(!row.count())
    {
        return "SAFE";
    }
    cur = ifnull(row.Premium_Current, 0);
    ren = row.Premium_Renewal;
    if(ren != null && cur > 0)
    {
        inc = ((ren - cur) / cur) * 100;
        if(inc > 15)
        {
            return "CRITICAL";
        }
        if(inc >= 5)
        {
            return "AT_RISK";
        }
    }
    if(row.Expiration_Date == null)
    {
        return "SAFE";
    }
    days = daysBetween(zoho.currentdate, row.Expiration_Date);
    if(days <= 30)
    {
        return "CRITICAL";
    }
    if(days <= 90)
    {
        return "AT_RISK";
    }
    return "SAFE";
}

```

```

map thisapp.recon.evaluateQueueStale(int queueID)
{
    out = Map();
    out.put("stale", false);
    out.put("reason", "");
    row = Renewal_Queue[ID == queueID];
    if(!row.count())
    {
        return out;
    }
    pol = Policy_Master[ID == row.Policy];
    pstat = thisapp.recon.normalizeStatus(ifnull(pol.Policy_Status, ""));
    openish = {"Open", "In Progress", "Waiting Client", "Waiting Carrier", "Stale"};
    if(pstat == "Cancelled" || pstat == "Flat Cancel" || pstat == "Rewritten" || pstat == "Renewed" ||
pstat == "Non-Renewed")
    {
        if(openish.contains(row.Queue_Status))
        {
            out.put("stale", true);
            out.put("reason", "Policy_Status=" + pstat + " but queue still " +
row.Queue_Status);
            return out;
        }
    }
    if(pol.Expiration_Date != null && row.Expiration_Date != null && pol.Expiration_Date !=

```

```

row.Expiration_Date)
{
    out.put("stale", true);
    out.put("reason", "Queue Expiration_Date != Policy_Master Expiration_Date");
    return out;
}
ageDays = 0;
if(row.Last_Touched_At != null)
{
    ageDays = daysBetween(row.Last_Touched_At.toDate(), zoho.currentdate);
}
else
{
    ageDays = 999;
}
bucket = ifnull(row.Cadence_Bucket, thisapp.recon.cadenceBucket(row.Expiration_Date));
qstat = ifnull(row.Queue_Status, "");
if((bucket == "30" || bucket == "Past Due") && (qstat == "Open" || qstat == "In Progress") &&
ageDays > 7)
{
    out.put("stale", true);
    out.put("reason", "Cadence " + bucket + " untouched " + ageDays + " days");
    return out;
}
if(bucket == "60" && qstat == "Open" && ageDays > 14)
{
    out.put("stale", true);
    out.put("reason", "Cadence 60 Open untouched " + ageDays + " days");
    return out;
}
return out;
}

map thisapp.recon.evaluateQueueStaleFromPolicy(int policyID)
{
    q = Renewal_Queue[Policy == policyID && Dismissed == false];
    out = Map();
    out.put("stale", false);
    out.put("reason", "");
    if(!q.count())
    {
        return out;
    }
    return thisapp.recon.evaluateQueueStale(q.ID);
}

```

08_crm_pull.dg

```

// Zoho CRM read (Phase 4). Connection name: zoho_crm
// Custom module API name may be "Policies" or "CustomModuleX" – confirm in
// Settings → APIs → API Names after the CRM field pack is installed
// (docs/zoho/fields_policies.csv).
//
// NEVER create Accounts. NEVER update CRM unless approval gate passes.

void thisapp.recon.pullCRM(int policyID)
{
    rec = Policy_Master[ID == policyID];
    if(!rec.count())
    {
        return;
    }
    guid = ifnull(rec.NowCerts_Policy_GUID, "");
    pnum = ifnull(rec.Policy_Number, "");
    crmRow = null;
    if(guid != "")
    {
        crit = "(NowCerts_Policy_GUID>equals:" + guid + ")";
    }
}

```

```

        resp = zoho.crm.searchRecords("Policies", crit, 1, 2);
        if(resp.size() == 1)
        {
           .crmRow = resp.get(0);
        }
        else if(resp.size() > 1)
        {
            rec.Sync_Status="Error";
            rec.Notes=ifnull(rec.Notes, "") + "\n[recon] duplicate CRM Policies for GUID " +
guid;
            return;
        }
    }
    if(crmRow == null && pnum != "")
    {
        crit2 = "(Policy_Number>equals:" + pnum + ")";
        resp2 = zoho.crm.searchRecords("Policies", crit2, 1, 2);
        if(resp2.size() == 1)
        {
           .crmRow = resp2.get(0);
        }
        else if(resp2.size() > 1)
        {
            rec.Sync_Status="Error";
            rec.Notes=ifnull(rec.Notes, "") + "\n[recon] duplicate CRM Policies for number " +
pnum;
            return;
        }
    }
    if(crmRow == null)
    {
        return;
    }
    rec.CRM_Policy_ID=crmRow.get("id");
    acct = crmRow.get("Account_Name");
    if(acct != null && acct.get("id") != null)
    {
        rec.CRM_Account_ID=acct.get("id");
    }
    rec.CRM_Status=thisapp.recon.normalizeStatus(ifnull(crmRow.get("Policy_Status"), ""));
    if(crmRow.get("Premium") != null)
    {
        rec.CRM_Premium=crmRow.get("Premium");
    }
    if(crmRow.get("Modified_Time") != null)
    {
        rec.Last_CRM_Modified=crmRow.get("Modified_Time").toTime("yyyy-MM-dd'T'HH:mm:ss");
    }
    rec.Last_Synced=zoho.currenttime;
    if(ifnull(rec.Sync_Status, "") != "Pending")
    {
        rec.Sync_Status="Synced";
    }
}

void thisapp.recon.pullCRMBatch()
{
    rows = Policy_Master[Active == true || Policy_Tier == "A" || Policy_Tier == "B"];
    for each rec in rows
    {
        thisapp.recon.pullCRM(rec.ID);
    }
}

// GATED WRITE – do not call from schedules.
// Preconditions: Approved_To_Push, Approved_By, Approved_At within 24h,
// latest Policy_Audit.Recommended_Payload non-empty.

map thisapp.recon.pushCRMIfApproved(int policyID)
{

```



```

out = Map();
out.put("ok", false);
rec = Policy_Master[ID == policyID];
if(!rec.count())
{
    out.put("error", "not found");
    return out;
}
if(rec.Approved_To_Push != true || rec.Approved_By == null || rec.Approved_At == null)
{
    out.put("error", "approval gate failed");
    return out;
}
ageHours = (zoho.currenttime.toLong() - rec.Approved_At.toLong()) / 3600000;
if(ageHours > 24)
{
    out.put("error", "approval older than 24h");
    return out;
}
if(ifnull(rec.CRM_Policy_ID, "") == "")
{
    out.put("error", "no CRM_Policy_ID – create drafts are human-only");
    return out;
}
aud = Policy_Audit[Policy == policyID] sort by Run_At desc range from 0 to 0;
payloadText = ifnull(aud.Recommended_Payload, "");
if(payloadText == "")
{
    out.put("error", "empty Recommended_Payload");
    return out;
}
// Expected payload: JSON map of CRM API names → values. Example:
// {"Policy_Status":"Active","Premium":1200.00}
payload = payloadText.toMap();
resp = zoho.crm.updateRecord("Policies", rec.CRM_Policy_ID, payload);
rec.Approved_To_Push=false;
rec.Sync_Status="Pending";
thisapp.recon.writeStatusHistory(policyID, rec.Policy_Status, rec.Policy_Status, "Creator");
out.put("ok", true);
out.put("crm_response", resp);
return out;
}

void thisapp.recon.pullCRM_all_modified_since()
{
    // Optional: zoho.crm.getRecords with Modified_Time criteria.
    // Implement after live API names are confirmed. Do not guess module names.
}

```

Appendix D — Seed acceptance (Phase 2)

TEST DATA ONLY. Import after Phase 2. Expected verdict and confidence must match.

Seed	Policy_Number	Expected verdict	Expected confidence
S01	RSG-TEST-S01	clean_match	100
S02	RSG-TEST-S02	duplicate_policy	70
S02b	RSG-TEST-S02	duplicate_policy	70
S03	RSG-TEST-S03	pending_sync	92
S04b	RSG-TEST-S04B	clean_match	100
S04	RSG-TEST-S04	rewrite_detected	100
S05	RSG-TEST-S05	status_mismatch	88
S06	RSG-TEST-S06	financial_discrepancy	82
S07	RSG-TEST-S07	missing_in_crm	90
S08	RSG-TEST-S08	missing_in_ams	75
S09	RSG-TEST-S09	stale_renewal_queue	90
S10	RSG-TEST-S10	stale_crm	100
S11	RSG-TEST-S11	cancel_reason_gap	88
S12	RSG-TEST-S12	lineage_orphan	88